

## 01\_collapse\_loop.sh

```
#!/bin/bash
WD=/scratch/unifesp/pgt/liriel.almodobar/puzzle
INPUT_DIR=$WD/bhrc_lai
#grep C $INPUT_DIR/*.sample > $WD/infos/individuos.txt
chr=$1
if [ ! -d "$WD/output_collapse/chr_${chr}" ]; then
    mkdir "$WD/output_collapse/chr_${chr}"
fi
while read p; do
    python3 $WD/scripts/collapse_ancestry.py \
        --rfmix
"$INPUT_DIR/BHRC_JOIN_RefHGDPNAT_1kgIBS_1kgYRI_chr${chr}.2.Viterbi.txt" \
    --snp_locations
"$INPUT_DIR/BHRC_JOIN_RefHGDPNAT_1kgIBS_1kgYRI_chr${chr}.snp_locations" \
    --fbk
"$INPUT_DIR/BHRC_JOIN_RefHGDPNAT_1kgIBS_1kgYRI_chr${chr}.2.ForwardBackward.t
xt.gz" \
    --fbk_threshold 0.9 \
    --ind "$p" \
    --ind_info "$INPUT_DIR/BHRC_JOIN_RefHGDPNAT_1kgIBS_1kgYRI.sample" \
    --pop_labels NAT,EUR,AFR \
    --chr $chr \
    --out "$WD/output_collapse/chr_${chr}/${p}"
done < $WD/infos/individuos.txt
```

## 02\_frag\_info\_per\_anc\_chr.sh

```
#!/bin/bash

WD=/home/yuri/liri/puzzle_sdumont

chr=$1

# Define the input files directory
INPUT_FILES=/home/yuri/liri/puzzle_sdumont/output_collapse/chr_${chr}

# Function to process states and ancestries
process_states_ancs() {
    local state=$1
    local label_lower=$2

    if [ "$state" = "rs_sp" ]; then
        grep -w "$label" "$INFOS/all_anc_${chr}.txt" | grep -v -E
"C20361_A|C20361_B|C20776_A|C20776_B" | awk '{print $0 ($1 ~ /^C1/ ? "\trs" : "\tsp")}' >
"$ANC_DIR/${label_lower}_${state}_${chr}.txt"
    elif [ "$state" = "rs" ]; then
```

```

        awk '{if ($6 == "rs") print $0}'
"$WD/rs_sp/${label_lower}/${label_lower}_rs_sp_${chr}.txt" >
"$ANC_DIR/${label_lower}_${state}_${chr}.txt"
    else
        awk '{if ($6 == "sp") print $0}'
"$WD/rs_sp/${label_lower}/${label_lower}_rs_sp_${chr}.txt" >
"$ANC_DIR/${label_lower}_${state}_${chr}.txt"
    fi
}

```

# Function to process a specific chromosome

```
process_chromosome() {
```

```
    local chr=$1
```

```
    local label=$2
```

```
    local ANC_DIR=$3
```

```

    awk -v chr=$chr -v OFS="\t" '{ if ($2 == chr) {print}}'
"$ANC_DIR/${label}_${state}_${chr}.txt" | sort -k3,3nb -k4,4nbr >
"$WD/${chr}_${label}_temp.txt"

```

```

    if [ -s "$WD/${chr}_${label}_temp.txt" ]; then
        awk '!seen[$3]++' "$WD/${chr}_${label}_temp.txt" | awk 'NR==1 {a=$4; printf
"%s\t%d\t%d\t%d\t%d\tNA\t%s\t%s\n", $1, $2, $3, $4, ($4-$3), $5, $6; next} {printf
"%s\t%d\t%d\t%d\t%d\t%d\t%s\t%s\n", $1, $2, $3, $4, ($4-$3), ($3 <= a) ? 0 : 1, $5, $6;
a=$4}' > "$ANC_DIR/chr_info_filt/chr_${chr}_${label}_${state}_filt.txt"

```

```

        awk 'NR==1 {a=$4; printf "%s\t%d\t%d\t%d\t%d\tNA\t%s\t%s\n", $1, $2, $3, $4, ($4-$3),
$5, $6; next} {printf "%s\t%d\t%d\t%d\t%d\t%d\t%s\t%s\n", $1, $2, $3, $4, ($4-$3), ($3 <= a)
? 0 : 1, $5, $6; a=$4}' "$WD/${chr}_${label}_temp.txt" >
"$ANC_DIR/chr_info_unfilt/chr_${chr}_${label}_${state}_unfilt.txt"
        echo "$ANC_DIR/chr_info_unfilt/chr_${chr}_${label}_${state}_unfilt.txt"
    fi
    rm "$WD/${chr}_${label}_temp.txt"
}

```

# Check if the 'infos' directory exists and create it if not

```
if [ ! -d "$WD/infos" ]; then
```

```
    mkdir "$WD/infos"
```

```
fi
```

# Set the 'infos' directory path

```
INFOS="$WD/infos"
```

# List files in the input directory and save the results to a file

```
ls "$INPUT_FILES" > "$WD/collapse_results_${chr}.txt"
```

```
echo "$WD/collapse_results_${chr}.txt"
```

```

# Process each file listed in 'collapse_results.txt'
while read collapse; do
    FILE_NAME=$(echo "$collapse" | head -n 1 | sed "s/.bed//g")
    cut -f1-4 "$INPUT_FILES/$collapse" | sed "s/^/$FILE_NAME\t/" >>
"$INFOS/all_anc_${chr}.txt"
done < "$WD/collapse_results_${chr}.txt"

# Remove the temporary file 'collapse_results.txt'
rm "$WD/collapse_results_${chr}.txt"

# Define an array of states
states=("rs_sp" "rs" "sp")

# Define an array of labels
labels=("NAT" "EUR" "AFR" "UNK")

# Process each state and label
for state in "${states[@]}"; do
    STATE_DIR=$WD/$state
    if [ ! -d "$STATE_DIR" ]; then
        mkdir "$STATE_DIR"
    fi

    for label in "${labels[@]}"; do
        label_lower="${label,,}"
        ANC_DIR="$STATE_DIR/$label_lower"
        echo "$ANC_DIR"
        # Create a directory for the label if it doesn't exist
        if [ ! -d "$ANC_DIR" ]; then
            mkdir "$ANC_DIR"
        fi

        if [ ! -d "$ANC_DIR/chr_info_filt" ]; then
            mkdir "$ANC_DIR/chr_info_filt"
        fi

        if [ ! -d "$ANC_DIR/chr_info_unfilt" ]; then
            mkdir "$ANC_DIR/chr_info_unfilt"
        fi

        # Process states and ancs
        process_states_ancs "$state" "$label_lower"

        process_chromosome "$chr" "$label_lower" "$ANC_DIR"
    done
done
rm $INFOS/all_anc_${chr}.txt

```

### 03\_format\_haps.sh

WD=/scratch/unifesp/pgt/liriel.almodobar/puzzle

HAPS\_SAMPLE\_DIR=\$WD/bhrc\_haps\_hg38

# Assigning arguments to variables

chr=\$1

#Functions

##Process .haps file

### Generate .haps with header

```
haps_header() {
    awk '{print $2}' $WD/haps_cols_${chr}.txt | tr '\n' '\t' > $WD/geno_columns_${chr}.txt
    paste $WD/infos/5_columns.txt $WD/geno_columns_${chr}.txt >
$WD/header_${chr}.txt #5_columns should go to info
    rm $WD/geno_columns_${chr}.txt
    rm $WD/haps_cols_${chr}.txt
}
```

### Replace 1 and 0 by the actual allele

```
get_allele() {
    awk '{
        fourth_col=$4
        fifth_col=$5
        modified_line=$1"\t"$2"\t"$3"\t"fourth_col"\t"fifth_col
        for (i=6; i<=NF; i++) {
            if ($i == 0) {
                modified_line = modified_line"\t"fourth_col
            } else if ($i == 1) {
                modified_line = modified_line"\t"fifth_col
            }
        }
        print modified_line
    }' $subset_haps_file > $WD/alleles_${chr}.txt
}
```

```
join_header_allele(){
    cat $WD/header_${chr}.txt $WD/alleles_${chr}.txt > $header_allele_file
    rm $WD/alleles_${chr}.txt
    rm $WD/header_${chr}.txt
}
```

```
get_header_allele(){
```

```

        local subset_haps_file=$1
        local header_allele_file=$2
        haps_header
        get_allele
        join_header_allele
    }

```

#Main script

# Subset haps for my sample

##Create necessary dirs for future steps

```

if [ ! -d "$WD/haps_bhrc_children" ]; then
    mkdir "$INPUT_DIR/haps_bhrc_children"
fi

```

```

tail -n +3 "$WD/bhrc_haps_hg38/INPD_hg38_${chr}.sample" | grep -n C | sed s#:0##g | sed
's#_[^ ]*##g' | awk '{ print $1, $2}' | sed "p" |awk 'NR%2{suffix="_A"} !(NR%2){suffix="_B"}
{print $0 suffix}' | awk '{if (NR % 2 == 0) $1 = ($1 * 2) + 5 ; else $1 = ($1 * 2) - 1 + 5} 1' >
$WD/haps_cols_${chr}.txt
awk '{print $1}' $WD/haps_cols_${chr}.txt > $WD/haps_indexes_${chr}

```

## Declare an empty array  
indexes=()

### Read the file line by line and populate the array

```

while IFS= read -r line; do
    indexes+=("$line")
done < $WD/haps_indexes_${chr}

```

```

gunzip "$WD/bhrc_haps_hg38/INPD_hg38_${chr}.haps"
awk -v OFS="\t" -v indexes="${indexes[*]}" '{split(indexes, arr, " "); for (i in arr) printf "%s ",
$arr[i]; print ""}' "$WD/bhrc_haps_hg38/INPD_hg38_${chr}.haps" >
$WD/ind_cols_haps_${chr}
awk '{print $1,$2,$3,$4,$5}' "$WD/bhrc_haps_hg38/INPD_hg38_${chr}.haps" >
$WD/first_haps_${chr}
paste $WD/first_haps_${chr} $WD/ind_cols_haps_${chr} >
"$WD/haps_bhrc_children/bhrc_hg38_children_chr_${chr}.haps"

```

```

rm $WD/haps_indexes_${chr}
rm $WD/first_haps_${chr}
rm $WD/ind_cols_haps_${chr}

```

```

get_header_allele "$WD/haps_bhrc_children/bhrc_hg38_children_chr_${chr}.haps"
$WD/infos/haps_geno_header_${chr}.txt

```

#### 04\_allele\_freq.sh

```
WD=/home/yuri/liri/puzzle_sdumont
#HAPS_SAMPLE_DIR=$WD/bhrc_haps_hg38

# Check if all three arguments are provided
if [ $# -ne 3 ]; then
    echo "Usage: $0 <state> <anc> <chr>"
    exit 1
fi

# Assigning arguments to variables
state=$1
anc=$2
chr=$3

#Obtaining allele frequency through processed haps file

INPUT_FILE="$WD/$state/$anc/chr_info_unfilt/chr_${chr}_${anc}_${state}_unfilt.txt"
INPUT_DIR="$WD/$state/$anc/chr_info_unfilt"

##Create necessary dirs for future steps
if [ ! -d "$INPUT_DIR/count_info" ]; then
    mkdir "$INPUT_DIR/count_info"
fi

if [ ! -d "$INPUT_DIR/seq_info" ]; then
    mkdir "$INPUT_DIR/seq_info"
fi

###Functions

####Find haps data in collapse intervals (merge collapse and haps)

find_vars_within_pos_range() {
    local pos_file="$1"
    local var_file="$2"
    local output_file="$3"
    while read -r id chrom initial_pos final_pos _ _ _; do
        ##### Search for matching positions in bim file
        index_id=$(awk -v id="$id" '{ for (i=1; i<=NF; i++) if ($i == id) { print i; exit } }'
"$var_file")
        if [ -n "$index_id" ]; then
            vars=$(awk -v OFS="\t" -v index_id="$index_id" -v ip="$initial_pos" -v
fp="$final_pos" -v id="$id" ' $3 >= ip && $3 <= fp { print $2 " ", $3 " ", $index_id } ' "$var_file" | tr
"\n" "\t")
            if [ -n "$vars" ]; then #testing only
```

```

        echo -e "$id\t$chrom\t$initial_pos\t$final_pos\t$vars" >> "$output_file"
    fi
fi
done < "$pos_file"
}

###Obtain allele frequencies

#### Count occurrences of var_allele combinations

count_var_allele(){
    count=$(grep -w "$var_allele" "$cohaps" | wc -l)
    echo -e "$var_allele\t$chr\t$count\t" | sed 's#,#\t#g' | awk -v OFS="\t" '{print $1, $4,
$2, $3, $5}' >> "$WD/count_var_allele_${state}_${anc}_${chr}.txt"
}

#### Get allele frequencies
allele_freq(){
    sum_var=$(grep "$var" "$WD/count_var_allele_${state}_${anc}_${chr}.txt" | awk '{
sum += $5 } END { print sum }')
    awk -v OFS="\t" -v var=$var -v sum_var=$sum_var '{if ($1 == var) {print $1, $2, $3,
$4, $5, sum_var, ($5/sum_var)*100}}' "$WD/count_var_allele_${state}_${anc}_${chr}.txt" >>
"$INPUT_DIR/count_info/freqs_chr_${chr}_${anc}_${state}.txt"
}

###Main script

find_vars_within_pos_range $INPUT_FILE "$WD/infos/haps_geno_header_${chr}.txt"
"$INPUT_DIR/seq_info/cohaps_chr_${chr}_${anc}_${state}.txt"

cohaps="$INPUT_DIR/seq_info/cohaps_chr_${chr}_${anc}_${state}.txt"

#### Getting variant_alleles combinations to search
awk '{for(i=5;i<=NF;i++) print $i}' "$cohaps" | sort -b | uniq >
"$WD/varal_info_entrada_${state}_${anc}_${chr}.txt"

while read var_allele; do
    count_var_allele "$var_allele"
done < "$WD/varal_info_entrada_${state}_${anc}_${chr}.txt"

####Getting variants to search
cut -f 1 "$WD/count_var_allele_${state}_${anc}_${chr}.txt" | sort -b | uniq >
"$WD/temp_input_${state}_${anc}_${chr}.txt"

while read var; do
    allele_freq $var
done < "$WD/temp_input_${state}_${anc}_${chr}.txt"

```

```
#rm "$WD/varal_info_entrada_${state}_${anc}_${chr}.txt"
#rm "$WD/count_var_allele_${state}_${anc}_${chr}.txt"
#rm "$WD/temp_input_${state}_${anc}_${chr}.txt"
```

## 05.1\_comp\_occur\_rs\_sp.R

```
# Load required libraries
```

```
library(dplyr)
```

```
library(tidyr)
```

```
occur_dir <- "/home/yuri/liri/puzzle/comp_occur/all/3dp/"
```

```
plot_dir <- "/home/yuri/liri/puzzle/rs_sp_plots/3dp/"
```

```
diff_all_chr <- c()
```

```
pos_all <- c()
```

```
chr_all <- c()
```

```
rsids_all <- c()
```

```
# Define the loop over chromosomes
```

```
for (chr in 1:22) {
```

```
  #Data processing
```

```
  ## NAT RS
```

```
  freqs_rs <-
```

```
  paste0("/home/yuri/liri/puzzle_sdumont/rs/nat/chr_info_unfilt/count_info/freqs_chr_", chr,
"_nat_rs.txt")
```

```
  count_nat_rs <- read.table(freqs_rs, h=F)
```

```
  count_nat_rs <- count_nat_rs[order(count_nat_rs[,3]),]
```

```
  count_nat_rs <- count_nat_rs[,c(1, 2,3,6)]
```

```
  count_nat_rs <- count_nat_rs[!duplicated(count_nat_rs),]
```

```
  colnames(count_nat_rs) <- c("rsid", "chrom", "bp", "score")
```

```
  count_nat_rs$chrom <- paste("chr", count_nat_rs$chrom, sep="")
```

```
  count_nat_rs$end <- c(count_nat_rs$bp[2:length(count_nat_rs$bp)] - 1,
```

```
count_nat_rs$bp[length(count_nat_rs$bp)] + 1)
```

```
  count_nat_rs <- count_nat_rs[,c(1,2,3,5,4)]
```

```
  count_nat_rs$score <- count_nat_rs$score*0.116
```

```
  colnames(count_nat_rs) <- c("rsid", "chrom", "start", "end", "score")
```

```
  ## NAT SP
```

```
  freqs_sp <-
```

```
  paste0("/home/yuri/liri/puzzle_sdumont/sp/nat/chr_info_unfilt/count_info/freqs_chr_", chr,
"_nat_sp.txt")
```

```
  count_nat_sp <- read.table(freqs_sp, h=F)
```

```
  count_nat_sp <- count_nat_sp[order(count_nat_sp[,3]),]
```

```
  count_nat_sp <- count_nat_sp[,c(1,2,3,6)]
```

```
  count_nat_sp <- count_nat_sp[!duplicated(count_nat_sp),]
```

```
  colnames(count_nat_sp) <- c("rsid", "chrom", "bp", "score")
```

```
  count_nat_sp$chrom <- paste("chr", count_nat_sp$chrom, sep="")
```



```

count_nat_sp$end <- c(count_nat_sp$bp[2:length(count_nat_sp$bp)] - 1,
count_nat_sp$bp[length(count_nat_sp$bp)] + 1)
count_nat_sp <- count_nat_sp[,c(1,2,3,5,4)]
count_nat_sp$score <- count_nat_sp$score*0.156
colnames(count_nat_sp) <- c("rsid", "chrom", "start", "end", "score")

## Check position indexing
if (sum(count_nat_rs$start - count_nat_sp$start) != 0) {
  ### Find indices of the missing rows in count_nat_sp and count_nat_rs
  missing_in_rs_indices <- which(!(count_nat_sp$start %in% count_nat_rs$start)) # Indices
of rows in count_nat_sp not in count_nat_rs
  missing_in_sp_indices <- which(!(count_nat_rs$start %in% count_nat_sp$start)) # Indices
of rows in count_nat_rs not in count_nat_sp

  ### Add missing rows from count_nat_sp to count_nat_rs
  if (length(missing_in_rs_indices) > 0) {
    missing_rows_rs <- count_nat_sp[missing_in_rs_indices, ]
    missing_rows_rs$score <- 0 # Set score to 0 for missing rows
    count_nat_rs <- rbind(count_nat_rs, missing_rows_rs)
  }

  ### Add missing rows from count_nat_rs to count_nat_sp
  if (length(missing_in_sp_indices) > 0) {
    missing_rows_sp <- count_nat_rs[missing_in_sp_indices, ]
    missing_rows_sp$score <- 0 # Set score to 0 for missing rows
    count_nat_sp <- rbind(count_nat_sp, missing_rows_sp)
  }

  ### Ensure both data frames are sorted by position (start)
  count_nat_rs <- count_nat_rs[order(count_nat_rs$start), ]
  count_nat_sp <- count_nat_sp[order(count_nat_sp$start), ]

  ### Recalculate positions and ranges
  pos <- count_nat_rs$start # Positions (after ensuring consistency)
  pos_mb <- pos / 1e6      # Positions in Mb

  ### Minimum and maximum scores
  min_occur <- min(c(count_nat_rs$score, count_nat_sp$score))
  max_occur <- max(c(count_nat_rs$score, count_nat_sp$score))

} else {
  pos <- count_nat_rs$start
  pos_mb <- pos / 1e6
  min_occur <- min(c(count_nat_rs$score, count_nat_sp$score))
  max_occur <- max(c(count_nat_rs$score, count_nat_sp$score))
}

```

```

chr_save <- rep(chr, length(pos))
chr_all <- c(chr_all, chr_save)

pos_all <- c(pos_all, pos)
rsids <- count_nat_rs$rsid
rsids_all <- c(rsids_all, rsids)
occur_rs <- (count_nat_rs$score)
occur_sp <- (count_nat_sp$score)

#### Differences between states
diff <- occur_rs - occur_sp
diff_all_chr <- c(diff_all_chr, diff)

# Detect differences

diff_mean <- mean(diff)
diff_sd <- sd(diff)

plot_data_chr <- list(
  pos_mb = pos_mb,
  occur_rs = occur_rs,
  occur_sp = occur_sp,
  chr = chr,
  col_rs = 'blue',
  col_sp = 'red',
  ylab = "Weighted occurrences",
  xlab = paste("chr ", chr, " (Mb)"),
  ylim = c(0, max(occur_rs, occur_sp) + 5),
  xaxt = "n",
  bty = "n",
  legend_pos = "top",
  legend_labels = c("RS", "SP"),
  legend_fill = c("blue", "red"),
  axis_x_values = c(1, seq(20, max(pos_mb) + 20, by = 20)),
  axis_x_labels = c(1, seq(20, max(pos_mb) + 20, by = 20))
)

assign(paste0("plot_data_chr_", chr), plot_data_chr)
}

## All chrs

diff_mean_all_chr <- mean(diff_all_chr)
diff_sd_all_chr <- sd(diff_all_chr)
different_all_chr <- abs(diff_all_chr - diff_mean_all_chr) > 3 * diff_sd_all_chr
different_all_chr <- as.numeric(different_all_chr)
diff_sp_gt_rs_all_chr <- which(diff_all_chr < 0 & different_all_chr == 1)
diff_rs_gt_sp_all_chr <- which(diff_all_chr > 0 & different_all_chr == 1)

```

```
diff_pos_sp_gt_rs_all_chr <- pos_all[diff_sp_gt_rs_all_chr]
diff_pos_rs_gt_sp_all_chr <- pos_all[diff_rs_gt_sp_all_chr]
diff_pos_all_chr <- pos_all[as.logical(different_all_chr)]
```

```
diff_snp_sp_gt_rs_all_chr <- rsids_all[diff_sp_gt_rs_all_chr]
diff_snp_rs_gt_sp_all_chr <- rsids_all[diff_rs_gt_sp_all_chr]
diff_snp_all_chr <- rsids_all[as.logical(different_all_chr)]
```

```
diff_chr_sp_gt_rs_all_chr <- chr_all[diff_sp_gt_rs_all_chr]
diff_chr_rs_gt_sp_all_chr <- chr_all[diff_rs_gt_sp_all_chr]
diff_chr_all_chr <- chr_all[as.logical(different_all_chr)]
```

```
all_chr_rs_gt_sp_pos <- data.frame(
  chr = diff_chr_rs_gt_sp_all_chr,
  position = diff_pos_rs_gt_sp_all_chr,
  rsid = diff_snp_rs_gt_sp_all_chr
)
```

```
all_chr_sp_gt_rs_pos <- data.frame(
  chr = diff_chr_sp_gt_rs_all_chr,
  position = diff_pos_sp_gt_rs_all_chr,
  rsid = diff_snp_sp_gt_rs_all_chr
)
```

```
all_chr_diff_pos <- data.frame(
  chr = diff_chr_all_chr,
  position = diff_pos_all_chr,
  rsid = diff_snp_all_chr
)
```

```
write.csv(all_chr_diff_pos, paste0(occur_dir, "different_all_chr.csv"), row.names = FALSE,
quote = FALSE)
```

```
write.csv(all_chr_rs_gt_sp_pos, paste0(occur_dir, "diff_rs_gt_sp_all_chr.csv"), row.names =
FALSE, quote = FALSE)
```

```
write.csv(all_chr_sp_gt_rs_pos, paste0(occur_dir, "diff_sp_gt_rs_all_chr.csv"), row.names =
FALSE, quote = FALSE)
```

```
chromosomes_with_differences <- unique(all_chr_diff_pos$chr)
is_different_all_chr <- data.frame(chr_all, different_all_chr)
```

```
for (chr in chromosomes_with_differences) {
  # Obtain differing_positions vector
  is_different_chr <- subset(is_different_all_chr, chr_all == chr)
  is_different <- is_different_chr$different_all_chr
  # Construct the variable name for the stored plot data
```

```

plot_data_var <- paste0("plot_data_chr_", chr)

# Ensure the plot data exists
if (exists(plot_data_var, envir = .GlobalEnv)) {
  # Retrieve the stored plot data
  plot_data <- get(plot_data_var, envir = .GlobalEnv)

  scaling_factor <- max(c(plot_data$occur_rs, plot_data$occur_sp))
  png(paste0(plot_dir, "chr_", chr, "_rs_sp_all", ".png"), width = 2400, height = 1800, res =
300)
  # First, draw the black and green lines (so they appear behind)
  plot(plot_data$pos_mb, is_different * scaling_factor, type = 'l', col = 'darkgray',
      ylab = "Ocorrências ponderadas", xlab = paste("chr ", chr, " (Mb)", bty = "n", xaxt = "n",
ylim = c(0, max(plot_data$occur_rs, plot_data$occur_sp) + 5))
  # Then, draw the blue and red lines
  lines(plot_data$pos_mb, plot_data$occur_rs, col = 'blue')
  lines(plot_data$pos_mb, plot_data$occur_sp, col = 'red')

  # Add legend and axis as usual
  legend(x = "top", legend = c("RS", "SP", "Diferente"), fill = c("blue", "red", "darkgray"), ncol
= 3, bty = "n")
  x_values <- c(1, seq(20, max(plot_data$pos_mb) + 20, by = 20))
  x_labels <- x_values
  axis(1, at = x_values, labels = x_labels)
  dev.off()
}
}

chrs_2_more_regions <- c(4,18)
for (chr in chrs_2_more_regions) {
  all_chr_pos_diff <- data.frame(chr_all, pos_all, different_all_chr)
  chr_all_pos_diff <- subset(all_chr_pos_diff, chr_all == chr)
  assign(paste0("chr_", chr, "_pos_diff"), chr_all_pos_diff)
}
diff_4 <- subset(chr_4_pos_diff, pos_all >= 88535068)
diff_4_2 <- subset(diff_4, pos_all > 89206177 & pos_all <= 90635324)
diff_18 <- subset(chr_18_pos_diff, pos_all >= 69472017)
diff_18_2 <- subset(diff_18, pos_all > 69515429 & pos_all <= 72514646)
chr_4_rsids_region1 <- subset(all_chr_diff_pos, chr == 4 & position >= 88535068 & position
<= 89206177)
chr_4_rsids_region2 <- subset(all_chr_diff_pos, chr == 4 & position >= 90394425 & position
<= 90635324)
chr_18_rsids_region1 <- subset(all_chr_diff_pos, chr == 18 & position >= 69472017 &
position <= 69515429)
chr_18_rsids_region2 <- subset(all_chr_diff_pos, chr == 18 & position >= 70206844 &
position <= 72514646)

```

## 05.2\_comp\_allele\_freq\_rs\_sp.R

```

library(dplyr)
library(tidyr)

# Directories
rs_dir <- "/home/yuri/liri/puzzle_sdumont/rs/nat/chr_info_unfilt/count_info"
sp_dir <- "/home/yuri/liri/puzzle_sdumont/sp/nat/chr_info_unfilt/count_info"
allele_freq_dir <- "/home/yuri/liri/puzzle/comp_allele_freq/"

# Initialize empty data frames to store results for all chromosomes
rsvars_only_combined <- data.frame()
spvars_only_combined <- data.frame()
rssp_comm_combined <- data.frame()
rs_all <- data.frame()

# Loop through chromosomes 1 to 22
for (chr in 1:22) {
  # File paths for current chromosome
  rs_file <- file.path(rs_dir, paste0("freqs_chr_", chr, "_nat_rs.txt"))
  sp_file <- file.path(sp_dir, paste0("freqs_chr_", chr, "_nat_sp.txt"))

  # Read and filter data
  rs <- read.table(rs_file, sep = "\t", header = FALSE)
  sp <- read.table(sp_file, sep = "\t", header = FALSE)

  colnames(rs) <- c("SNP", "CHR", "BP", "allele", "count_rs", "total_rs", "freq_rs")
  colnames(sp) <- c("SNP", "CHR", "BP", "allele", "count_sp", "total_sp", "freq_sp")

  rs <- subset(rs, total_rs > 10)
  sp <- subset(sp, total_sp > 10)

  rs_all <- bind_rows(rs_all, rs)

  # Extract variables of interest
  rsvars <- rs[, c("CHR", "SNP", "allele", "freq_rs")]
  spvars <- sp[, c("CHR", "SNP", "allele", "freq_sp")]

  colnames(rsvars) <- c("chr", "var", "allele", "freq")
  colnames(spvars) <- c("chr", "var", "allele", "freq")

  # Common variants and frequency difference
  rssp_comm <- inner_join(rsvars, spvars, by = c("chr", "var", "allele"), suffix = c(".rsvars", ".spvars"))

  rssp_comm_combined <- bind_rows(rssp_comm_combined, rssp_comm)

  # Get rsvars_only and spvars_only
  rsvars_only <- anti_join(rsvars, spvars, by = c("chr", "var", "allele"))
  spvars_only <- anti_join(spvars, rsvars, by = c("chr", "var", "allele"))
}

```

```

# Append to combined data frames
rsvars_only_combined <- bind_rows(rsvars_only_combined, rsvars_only)
spvars_only_combined <- bind_rows(spvars_only_combined, spvars_only)
}

rssp_comm_combined <- rssp_comm_combined %>%
  mutate(allele_freq_diff_rssp = round(abs(freq.rsvars - freq.spvars), digits = 2),
  allele_freq_diff = round(freq.rsvars - freq.spvars, digits = 2))
diff_vars_all_chr <- subset(rssp_comm_combined, abs(allele_freq_diff_rssp -
mean(allele_freq_diff_rssp)) > (3 * sd(allele_freq_diff_rssp))) #only for all

max_diff_chr <- diff_vars_all_chr %>%
  dplyr::select(chr, var, allele, freq.rsvars, freq.spvars, allele_freq_diff_rssp) %>%
  group_by(chr) %>%
  slice_max(order_by = allele_freq_diff_rssp, n = 1) %>%
  dplyr::filter(row_number() %% 2 == 0) # Keep only even rows

# VEP input preparation
vcp <- inner_join(rs_all, diff_vars_all_chr, by = c("CHR" = "chr", "SNP" = "var", "allele")) #for
all: rs_all, diff_vars_all_chr
vcp <- vcp %>% mutate(rsids = ifelse(grepl("^rs", SNP), SNP, NA))
vcp <- vcp %>%
  mutate(SNP = paste(CHR, BP, sep = ".")) %>%
  dplyr::select(SNP, CHR, BP, allele, freq_rs, rsids)

subset_not_50 <- subset(vcp, freq_rs != 50)
subset_50_even <- subset(vcp, freq_rs == 50 & (1:nrow(vcp)) %% 2 == 0)
subset_50_odd <- subset(vcp, freq_rs == 50 & (1:nrow(vcp)) %% 2 == 1)

vcp_a1 <- subset_not_50 %>%
  group_by(SNP, CHR, BP) %>%
  slice_max(order_by = freq_rs) %>%
  ungroup() %>%
  bind_rows(subset_50_even)

vcp_a2 <- subset_not_50 %>%
  group_by(SNP, CHR, BP) %>%
  slice_min(order_by = freq_rs) %>%
  ungroup() %>%
  bind_rows(subset_50_odd)

colnames(vcp_a1) <- c("SNP", "CHR", "BP", "A2", "freq_A2", "rsids")
colnames(vcp_a2) <- c("SNP", "CHR", "BP", "A1", "freq_A1", "rsids")

vcp_all <- inner_join(vcp_a1, vcp_a2, by = c("SNP", "CHR", "BP", "rsids")) %>%
  dplyr::select(SNP, CHR, BP, A1, A2, rsids) %>%
  mutate(BP2 = BP, strand = 1, allele = paste(A2, A1, sep = "/")) %>%

```

```
dplyr::select(SNP, CHR, BP, BP2, strand, allele, rsids) %>%  
arrange(CHR, BP)
```

```
snv_diff_combined <- dplyr::filter(vep_all, !is.na(vep_all$rsids))
```

```
vep_all <- vep_all[,c(2,3,4,6,5)]
```

```
# Write combined data frames to files
```

```
write.csv(diff_vars_all_chr, paste0(allele_freq_dir, "diff_vars_all_chr.csv"), row.names =  
FALSE, quote = FALSE)
```

```
write.csv(max_diff_chr, paste0(allele_freq_dir, "max_diff_chr.csv"), row.names = FALSE,  
quote = FALSE)
```

```
write.csv(rsvars_only_combined, paste0(allele_freq_dir, "rsvars_only_combined.csv"),  
row.names = FALSE, quote = FALSE)
```

```
write.csv(spvars_only_combined, paste0(allele_freq_dir, "spvars_only_combined.csv"),  
row.names = FALSE, quote = FALSE)
```

```
write.csv(snv_diff_combined, paste0(allele_freq_dir, "snvs_diff_all.csv"), row.names = FALSE,  
quote = FALSE)
```

```
write.table(vep_all, paste0(allele_freq_dir, "vep_all.tsv"), row.names = FALSE, col.names =  
FALSE, quote = FALSE, sep = "\t")
```

```
06_enrichment_analysis.R
```

```
#Enrichment analysis
```

```
## libraries (see the ones that arent used)
```

```
library(dplyr)
```

```
library(rtracklayer)
```

```
library(fuzzyjoin)
```

```
library(tidyr)
```

```
library(biomaRt)
```

```
library(GenomicRanges)
```

```
library(clusterProfiler)
```

```
library(org.Hs.eg.db)
```

```
library(enrichplot)
```

```
library(KEGGREST)
```

```
library(AnnotationDbi)
```

```
library(DOSE)
```

```
## Functions
```

```
find_overlapping_genes <- function(snv_gr, gene_gr, analysis) {
```

```
  # Find overlaps between SNVs and genes
```

```
  overlap_genes <- findOverlaps(snv_gr, gene_gr)
```

```
  # Extract the gene indices and names
```

```
  overlap_gene_indices <- subjectHits(overlap_genes)
```

```

overlap_gene_names <- gene_gr$gene_name[overlap_gene_indices]
overlap_gene_ensembl_id <- gene_gr$gene_id[overlap_gene_indices]

# Combine gene names and Ensembl IDs into a DataFrame
overlap_gene_data <- DataFrame(
  gene_name = overlap_gene_names,
  ensembl_id = overlap_gene_ensembl_id
)

assign(paste0(analysis, "overlap_gene_data"), overlap_gene_data, envir = .GlobalEnv)

# Determine the gene info, default to Ensembl ID if gene name is NA or empty
gene_info <- ifelse(is.na(overlap_gene_names) | overlap_gene_names == "",
  overlap_gene_ensembl_id,
  overlap_gene_names)

# Return the list of unique gene names/IDs
genes_list_go <- unique(gene_info)

if(!is.null(genes_list_go)){
  return(genes_list_go)
} else {
  print("No overlapping genes")
}
}

get_entrez_ids <- function(overlap_gene_data) {

  # Retrieve Entrez IDs from Ensembl IDs
  entrez_mapping <- AnnotationDbi::select(org.Hs.eg.db, keys =
  overlap_gene_data$ensembl_id,
    columns = c("ENSEMBL", "ENTREZID"), keytype = "ENSEMBL")

  # Extract and return unique Entrez IDs
  entrez_ids <- unique(entrez_mapping$ENTREZID)
  if(!is.null(entrez_ids)){
    return(entrez_ids)
  } else {
    print("KEGG: no entrez ids")
  }
}

organize_enrichment_results <- function(enrichResult) {
  #Filters for significant results adjusted by multiple comparisons
  #Orders by gene ratio and then p.adjust

  ##Filter for significant results

```



```

results_df <- enrichResult@result
results_significant <- results_df[results_df$p.adjust < 0.05,]

##Order by p.adjust and gene ratio
if (!is.null(results_significant) && nrow(results_significant) != 0) {
  ### Split the GeneRatio values by the '/' character
  gene_ratio_parts <- strsplit(results_significant$GeneRatio, "/")
  results_significant$n_genes_input <- sapply(gene_ratio_parts, function(x)
as.numeric(x[2]))

  ### Perform the division and store the results
  results_significant$GeneRatio <- sapply(gene_ratio_parts, function(x) as.numeric(x[1]) /
as.numeric(x[2]))

  ### Now order by p.adjust (ascending) and then GeneRatio (descending)
  results_significant <- results_significant[order(results_significant$p.adjust,
-results_significant$GeneRatio), ]
  return(results_significant)
}
}

perform_go_enrichment <- function(genes_list) {
  # Perform GO enrichment analysis
  go_results <- enrichGO(
    gene = genes_list,
    keyType = "ENSEMBL",
    OrgDb = org.Hs.eg.db, # Human genome annotations
    ont = c("ALL")
  )
  # Organize the results
  all_go_results_df <- go_results@result
  go_results_df <- organize_enrichment_results(go_results)
  return(list(
    go_results_df = go_results_df,
    go_results = go_results,
    all_go_results_df = all_go_results_df
  ))
}

perform_kegg_enrichment <- function(entrez_ids, organism = "hsa", pvalue_cutoff = 0.05) {
  kegg_results <- enrichKEGG(
    gene = entrez_ids,
    organism = organism,
    pvalueCutoff = pvalue_cutoff
  )
  all_kegg_results_df <- kegg_results@result
  kegg_results_df <- organize_enrichment_results(kegg_results)
}

```

```

return(list(
  kegg_results_df = kegg_results_df,
  kegg_results = kegg_results,
  all_kegg_results_df = all_kegg_results_df
))
}

```

```

perform_dgn_enrichment <- function(snv_ids) {
  # SNP Disease-Gene Network Visualization
  dgn_results <- enrichDGNv(snv_ids)
  all_dgn_results_df <- dgn_results@result
  dgn_results_df <- organize_enrichment_results(dgn_results)
  return(list(
    dgn_results_df = dgn_results_df,
    dgn_results = dgn_results,
    all_dgn_results_df = all_dgn_results_df
  ))
}

```

```

generate_enrichment_plots <- function(enrich_result, output_dir, plot_title, plot_file,
enrich_type) {
  if (length(enrich_result) > 0) {
    # Bar plot
    barplot_file <- paste0(output_dir, "barplot_", plot_file, ".png")
    png(barplot_file, width = 1920, height = 2060, res = 150)
    print(barplot(enrich_result, showCategory = 20, title = plot_title, x = "GeneRatio"))
    dev.off()

    # Extract pathway data
    barplot_object <- print(barplot(enrich_result, showCategory = 20, title = plot_title, x =
"GeneRatio"))
    barplot_pathway_id <- barplot_object[["data"]][["ID"]]
    barplot_pathway_desc <- barplot_object[["data"]][["Description"]]
    barplot_pathways <- data.frame(barplot_pathway_id, barplot_pathway_desc)

    assign(paste0(enrich_type, "_barplot_object"), barplot_object, envir = .GlobalEnv)
    assign(paste0(enrich_type, "_barplot_pathways"), barplot_pathways, envir = .GlobalEnv)
    # Dot plot
    dotplot_file <- paste0(output_dir, "dotplot_", plot_file, ".png")
    png(dotplot_file, width = 1920, height = 1080, res = 150)
    print(dotplot(enrich_result, showCategory = 20, title = plot_title, size = "Count", x =
"GeneRatio"))
    dev.off()
  }
}

```

```

process_pathways_in_plot <- function(barplot_data, results_df, bg_ratio_col = "BgRatio") {

```

```

pathways_plot <- as.data.frame(barplot_data)
pathways_plot <- inner_join(pathways_plot, results_df,
                           by = c("barplot_pathway_id" = "ID",
                                "barplot_pathway_desc" = "Description"))
pathways_plot <- separate(pathways_plot, bg_ratio_col,
                          into = c("n_genes_pathway", "n_genes_homo_sapiens"), sep = "/")

return(pathways_plot)
}

```

### ## Directories

```

allele_freq_dir <- "/home/yuri/lori/puzzle/comp_allele_freq/"
freq_go_dir <- paste0(allele_freq_dir, "enrichment_analysis/go/")
freq_kegg_dir <- paste0(allele_freq_dir, "enrichment_analysis/kegg/")
freq_dgn_dir <- paste0(allele_freq_dir, "enrichment_analysis/dgn/")
occur_dir <- "/home/yuri/lori/puzzle/comp_occur/all/3dp/"
occur_go_dir <- paste0(occur_dir, "enrichment_analysis/go/")
occur_kegg_dir <- paste0(occur_dir, "enrichment_analysis/kegg/")
occur_dgn_dir <- paste0(occur_dir, "enrichment_analysis/dgn/")

```

### ### Create directories if they don't exist

```

dirs <- c(allele_freq_dir, freq_go_dir, freq_kegg_dir, freq_dgn_dir,
         occur_dir, occur_go_dir, occur_kegg_dir, occur_dgn_dir)

```

### #### Create directories recursively

```

sapply(dirs, function(d) if (!dir.exists(d)) dir.create(d, recursive = TRUE))

```

### ## Inputs

```

allele_freq <- read.csv(paste0(allele_freq_dir, "snvs_diff_all.csv"))
occur_rs_gt_sp <- read.csv(paste0(occur_dir, "diff_rs_gt_sp_all_chr.csv"))
occur_sp_gt_rs <- read.csv(paste0(occur_dir, "diff_sp_gt_rs_all_chr.csv"))
occur_diff_all <- read.csv(paste0(occur_dir, "different_all_chr.csv"))

```

### ## Retrieve gene information (chromosome, start, end positions, gene names)

```

ensembl = useMart("ensembl", dataset = "hsapiens_gene_ensembl", host =
"https://useast.ensembl.org")

```

```

gene_gr <- getBM(
  attributes = c("ensembl_gene_id", "external_gene_name", "chromosome_name",
"start_position", "end_position"),
  mart = ensembl
)

```

### ## Convert gene information into GenomicRanges object

```

gene_gr <- GRanges(
  seqnames = gene_gr$chromosome_name,
  ranges = IRanges(start = gene_gr$start_position, end = gene_gr$end_position),
  gene_id = gene_gr$ensembl_gene_id,
  gene_name = gene_gr$external_gene_name
)

## Enrichment analysis for differences in allele frequencies

snv_gr_freq <- GRanges(
  seqnames = allele_freq$CHR,
  ranges = IRanges(
    start = allele_freq$BP, # SNV positions
    end = allele_freq$BP # Same position for the end
  )
)

genes_list_freq <- find_overlapping_genes(snv_gr_freq, gene_gr, "freq_all_")
write.csv(genes_list_freq, paste0(freq_go_dir, "genes_all_freq.csv"), quote = FALSE,
row.names = FALSE)
genes_ensembl_ids <- unique(freq_all_overlap_gene_data$ensembl_id)
#### Convert to ENTREZ ids (necessary for KEGG enrichment)

entrez_ids_freq <- get_entrez_ids(freq_all_overlap_gene_data)
write.csv(entrez_ids_freq, paste0(freq_kegg_dir, "genes_kegg_all_freq.csv"), quote =
FALSE, row.names = FALSE)

#### GO

go_results_list_freq <- perform_go_enrichment(genes_ensembl_ids)
go_results_df_freq <- go_results_list_freq$go_results_df
go_results_freq <- go_results_list_freq$go_results
all_go_results_freq <- go_results_list_freq$all_go_results_df

if(!is.null(go_results_df_freq)) {
  write.csv(go_results_df_freq, paste0(freq_go_dir, "go_results_all_freq.csv"), quote =
FALSE, row.names = FALSE)

  #####Visualize the enrichment results
  title_go_freq <- paste0("Top enriched GO terms for high differences in allele freq.")
  plot_file_go_freq <- "go_results_freq"
  generate_enrichment_plots(go_results_freq, freq_go_dir, title_go_freq, plot_file_go_freq,
"go_freq")
} else {
  print("GO: No significant results")
}

#### KEGG

```

```

kegg_results_list_freq <- perform_kegg_enrichment(entrez_ids_freq)
kegg_results_df_freq <- kegg_results_list_freq$kegg_results_df
kegg_results_freq <- kegg_results_list_freq$kegg_results
all_kegg_results_freq <- kegg_results_list_freq$all_kegg_results_df

if(!is.null(kegg_results_df_freq)) {
  write.csv(kegg_results_df_freq, paste0(freq_kegg_dir, "kegg_results_freq_all.csv"), quote
= FALSE, row.names = FALSE)
  #####Visualize the enrichment results
  title_kegg_freq <- "Enriched KEGG pathways for high differences in allele freq."
  plot_file_kegg_freq <- "kegg_results_freq_all"
  generate_enrichment_plots(kegg_results_freq, freq_kegg_dir, title_kegg_freq,
plot_file_kegg_freq, "kegg_freq")
} else {
  print("KEGG: No significant results")
}

#### DGN

snv_ids_freq <- allele_freq$rsids

# Ensure there are SNP IDs to process
if (length(snv_ids_freq) > 0) {
  # SNP Disease-Gene Network Visualization
  dgn_results_list_freq <- perform_dgn_enrichment(snv_ids_freq)
  dgn_results_df_freq <- dgn_results_list_freq$dgn_results_df
  dgn_results_freq <- dgn_results_list_freq$dgn_results
  all_dgn_results_freq <- dgn_results_list_freq$all_dgn_results_df

  # Save results
  if (!is.null(dgn_results_freq)) {
    # Save as CSV
    write.csv(dgn_results_df_freq, paste0(freq_dgn_dir, "results_dgn_freq_all.csv"), quote =
FALSE, row.names = FALSE)

    title_dgn_freq <- "DGN Enrichment for high differences in allele freq."
    plot_file_dgn_freq <- "dgn_results_freq_all"
    generate_enrichment_plots(dgn_results_freq, freq_dgn_dir, title_dgn_freq,
plot_file_dgn_freq, "dgn_freq")
  } else {
    print("No significant results")
  }
} else {
  print("No SNP IDs available")
}

```

```
##Enrichment analysis for occurrences
```

```
comparisons <- c("different positions", "different positions (rs > sp)", "different positions (sp > rs)")
```

```
for (comp in comparisons) {  
  if (comp == "different positions") {  
    snv_gr_occur <- GRanges(  
      seqnames = occur_diff_all$chr, # Single chromosome (e.g., "1")  
      ranges = IRanges(  
        start = occur_diff_all$pos, # SNV positions  
        end = occur_diff_all$pos # Same position for the end  
      )  
    )  
  } else if (comp == "different positions (rs > sp)") {  
    snv_gr_occur <- GRanges(  
      seqnames = occur_rs_gt_sp$chr, # Single chromosome (e.g., "1")  
      ranges = IRanges(  
        start = occur_rs_gt_sp$pos, # SNV positions  
        end = occur_rs_gt_sp$pos # Same position for the end  
      )  
    )  
  } else if (comp == "different positions (sp > rs)") {  
    snv_gr_occur <- GRanges(  
      seqnames = occur_sp_gt_rs$chr, # Single chromosome (e.g., "1")  
      ranges = IRanges(  
        start = occur_sp_gt_rs$pos, # SNV positions  
        end = occur_sp_gt_rs$pos # Same position for the end  
      )  
    )  
  }  
}
```

```
comp_clean <- gsub("_+", "_", gsub(" ", "_", gsub("\\(|\\|)", "", gsub(">", "_", comp))))
```

```
### Find Genes Overlapping SNVs
```

```
genes_list_occur <- find_overlapping_genes(snv_gr_occur, gene_gr, "occur_")  
write.csv(genes_list_occur, paste0(occur_go_dir, "genes_", comp_clean, ".csv"), quote =  
FALSE, row.names = FALSE)  
genes_ensembl_ids <- unique(occur_overlap_gene_data$ensembl_id)
```

```
### Convert to ENTREZ ids (necessary for KEGG enrichment)
```

```
entrez_ids_occur <- get_entrez_ids(occur_overlap_gene_data)  
write.csv(entrez_ids_occur, paste0(occur_kegg_dir, "genes_kegg_", comp_clean, ".csv"),  
quote = FALSE, row.names = FALSE)
```

```

#### GO
go_results_list_occur <- perform_go_enrichment(genes_ensembl_ids)
go_results_df_occur <- go_results_list_occur$go_results_df
assign(paste0("go_results_df_occur_", comp_clean), go_results_df_occur)
all_go_results_occur <- go_results_list_occur$all_go_results_df
assign(paste0("all_go_results_occur_", comp_clean), all_go_results_occur)
go_results_occur <- go_results_list_occur$go_results
go_results_file <- paste0("go_results_3dp_", comp_clean)

if(!is.null(go_results_df_occur)) {
  write.csv(go_results_df_occur, paste0(occur_go_dir, go_results_file, ".csv"), quote =
FALSE, row.names = FALSE)

  #####Visualize the enrichment results
  title_go_occur <- paste0("Top enriched GO terms for ", comp)

  generate_enrichment_plots(go_results_occur, occur_go_dir, title_go_occur,
go_results_file, paste0("go_occur_", comp_clean))
}

#### KEGG

if(!is.null(entrez_ids_occur) && length(entrez_ids_occur) != 0) {
  kegg_results_list_occur <- perform_kegg_enrichment(entrez_ids_occur)
  kegg_results_df_occur <- kegg_results_list_occur$kegg_results_df
  assign(paste0("kegg_results_df_occur_", comp_clean), kegg_results_df_occur)
  all_kegg_results_occur <- kegg_results_list_occur$all_kegg_results_df
  assign(paste0("all_kegg_results_df_occur_", comp_clean), all_kegg_results_occur)
  kegg_results_occur <- kegg_results_list_occur$kegg_results
  kegg_results_file <- paste0("kegg_results_3dp_", comp_clean)
  if(!is.null(kegg_results_df_occur)) {
    write.csv(kegg_results_df_occur, paste0(occur_kegg_dir, kegg_results_file, ".csv"),
quote = FALSE, row.names = FALSE)
    #####Visualize the enrichment results
    title_kegg_occur <- paste0("Enriched KEGG pathways for ", comp)
    generate_enrichment_plots(kegg_results_occur, occur_kegg_dir, title_kegg_occur,
kegg_results_file, paste0("kegg_occur_", comp_clean))
  } else {
    print(paste0("KEGG: No significant results for comp ", comp))
  }
}

}

####DGN

##### Define comparisons

```

```

comparisons_snvs <- c("different positions snvs",
                     "different positions snvs (rs > sp)", "different positions snvs (sp > rs)")

for (comps in comparisons_snvs) {
  ##### Select appropriate SNP data based on the comparison
  if (comps == "different positions snvs") {
    snv_ids_occur <- occur_diff_all$rsid
  } else if (comps == "different positions snvs (rs > sp)") {
    snv_ids_occur <- occur_rs_gt_sp$rsid
  } else if (comps == "different positions snvs (sp > rs)") {
    snv_ids_occur <- occur_sp_gt_rs$rsid
  }

  comps_clean <- gsub("_+", "_", gsub(" ", "_", gsub("\\(|\\|)", "", gsub(">", "_", comps))))
  ##### Ensure there are SNP IDs to process
  if (length(snv_ids_occur) > 0) {
    dgn_results_list_occur <- perform_dgn_enrichment(snv_ids_occur)
    dgn_results_df_occur <- dgn_results_list_occur$dgn_results_df
    assign(paste0("dgn_results_df_occur_", comps_clean), dgn_results_df_occur)
    all_dgn_results_occur <- dgn_results_list_occur$all_dgn_results_df
    assign(paste0("all_dgn_results_df_occur_", comps_clean), all_dgn_results_occur)
    dgn_results_occur <- dgn_results_list_occur$dgn_results
    if (!is.null(dgn_results_occur)) {
      # Save as CSV
      dgn_results_file <- paste0("results_dgn_3dp_", comps_clean, ".csv")
      write.csv(dgn_results_df_occur, paste0(occur_dgn_dir, dgn_results_file), quote = FALSE,
row.names = FALSE)
      plot_file <- paste0("dgn_3dp_", comps_clean)
      dgn_title_occur <- paste0("DGN Enrichment for ", comps)
      generate_enrichment_plots(dgn_results_occur, occur_dgn_dir, dgn_title_occur, plot_file,
paste0("dgn_occur_", comps_clean))
    } else {
      print(paste0("SNP-DGNV: No significant results for comp ", comps))
    }
  } else {
    print(paste0("No SNP IDs available for comp", comps))
  }
}

```

## ## Detailed results

```

pathways_in_plot <- list(
  go_freq = list(go_freq_barplot_pathways, go_results_df_freq,
c(1,2,3,4,5,6,11,12,7,8,9,10)),
  kegg_freq = list(kegg_freq_barplot_pathways, kegg_results_df_freq,
c(1,2,3,4,5,12,13,6,7,8,9,10,11)),
  dgn_freq = list(dgn_freq_barplot_pathways, dgn_results_df_freq,
c(1,2,3,4,5,10,11,6,7,8,9)),

```



```

go_occur_rs_gt_sp = list(go_occur_different_positions_rs_sp_barplot_pathways,
go_results_df_occur_different_positions_rs_sp, c(1,2,3,4,5,6,11,12,7,8,9,10)),
go_occur_sp_gt_rs = list(go_occur_different_positions_sp_rs_barplot_pathways,
go_results_df_occur_different_positions_sp_rs, c(1,2,3,4,5,6,11,12,7,8,9,10)),
go_occur_diff = list(go_occur_different_positions_barplot_pathways,
go_results_df_occur_different_positions, c(1,2,3,4,5,6,11,12,7,8,9,10)),
dgn_occur_diff = list(dgn_occur_different_positions_snvs_barplot_pathways,
dgn_results_df_occur_different_positions_snvs, c(1,2,3,4,5,10,11,6,7,8,9)),
dgn_occur_rs_gt_sp = list(dgn_occur_different_positions_snvs_rs_sp_barplot_pathways,
dgn_results_df_occur_different_positions_snvs_rs_sp, c(1,2,3,4,5,10,11,6,7,8,9)),
dgn_occur_sp_gt_rs = list(dgn_occur_different_positions_snvs_sp_rs_barplot_pathways,
dgn_results_df_occur_different_positions_snvs_sp_rs, c(1,2,3,4,5,10,11,6,7,8,9))
)

```

```

pathways_in_plot_processed <- Map(function(barplot, results, order) {
  process_pathways_in_plot(barplot, results)[, order]
}, lapply(pathways_in_plot, `[`, 1), lapply(pathways_in_plot, `[`, 2), lapply(pathways_in_plot,
`[`, 3))

```

```

pathways_go_plot_freq <- pathways_in_plot_processed$go_freq
pathways_kegg_plot_freq <- pathways_in_plot_processed$kegg_freq
pathways_dgn_plot_freq <- pathways_in_plot_processed$dgn_freq
pathways_go_plot_occur_rs_sp <- pathways_in_plot_processed$go_occur_rs_gt_sp
pathways_go_plot_occur_sp_rs <- pathways_in_plot_processed$go_occur_sp_gt_rs
pathways_go_plot_occur_diff <- pathways_in_plot_processed$go_occur_diff
pathways_dgn_plot_occur_diff <- pathways_in_plot_processed$dgn_occur_diff
pathways_dgn_plot_occur_rs_sp <- pathways_in_plot_processed$dgn_occur_rs_gt_sp
pathways_dgn_plot_occur_sp_rs <- pathways_in_plot_processed$dgn_occur_sp_gt_rs

```

```

write.csv(pathways_go_plot_freq, paste0(freq_go_dir, "pathways_go_plot_freq.csv"),
row.names = FALSE, quote = FALSE)
write.csv(pathways_kegg_plot_freq, paste0(freq_kegg_dir, "pathways_kegg_plot_freq.csv"),
row.names = FALSE, quote = FALSE)
write.csv(pathways_dgn_plot_freq, paste0(freq_dgn_dir, "pathways_dgn_plot_freq.csv"),
row.names = FALSE, quote = FALSE)
write.csv(pathways_go_plot_occur_rs_sp, paste0(occur_go_dir,
"pathways_go_plot_occur_rs_sp.csv"), row.names = FALSE, quote = FALSE)
write.csv(pathways_dgn_plot_occur_diff, paste0(occur_dgn_dir,
"pathways_dgn_plot_occur_diff.csv"), row.names = FALSE, quote = FALSE)
write.csv(pathways_dgn_plot_occur_rs_sp, paste0(occur_dgn_dir,
"pathways_dgn_plot_occur_rs_sp.csv"), row.names = FALSE, quote = FALSE)
write.csv(pathways_dgn_plot_occur_sp_rs, paste0(occur_dgn_dir,
"pathways_dgn_plot_occur_sp_rs.csv"), row.names = FALSE, quote = FALSE)
write.csv(pathways_go_plot_occur_diff, paste0(occur_dgn_dir,
"pathways_go_plot_occur_diff.csv"), row.names = FALSE, quote = FALSE)
write.csv(pathways_go_plot_occur_rs_sp, paste0(occur_dgn_dir,
"pathways_go_plot_occur_rs_sp.csv"), row.names = FALSE, quote = FALSE)

```

```
write.csv(pathways_go_plot_occur_sp_rs, paste0(occur_dgn_dir,
"pathways_go_plot_occur_sp_rs.csv"), row.names = FALSE, quote = FALSE)
```

## 07\_eqtl.R

```
library(arrow)
library(dplyr)
library(tidyr)
library(ggplot2)
```

```
#preparing eQTL data from GTEx
```

```
eqtls_list <- list.files("/home/yuri/Downloads/GTEx_Analysis_v10_eQTL_updated/", pattern =
"*.parquet", full.names = TRUE)
eqtls <- lapply(eqtls_list, read_parquet)
eqtls <- lapply(eqtls, as.data.frame)
eqtls_vars_pos <- sapply(eqtls, separate, variant_id, into = c("chr", "pos"), sep = "_", remove
= FALSE)
eqtls_vars_pos <- lapply(seq_len(ncol(eqtls_vars_pos)), function(i) {
  data.frame(chr = eqtls_vars_pos["chr", i], pos = eqtls_vars_pos["pos", i])
})
eqtls_vars_pos <- lapply(eqtls_vars_pos, function(df) {
  df$chr <- gsub("chr", "", df$chr)
  return(df)
})
eqtls_vars_pos <- lapply(eqtls_vars_pos, function(df) {
  df$chr <- as.numeric(df$chr) # Apply gsub to resto column
  df
})
eqtls_vars_pos <- lapply(eqtls_vars_pos, function(df) {
  df$pos <- as.numeric(df$pos) # Apply gsub to resto column
  df
})
```

```
#Occurrences RS > SP
```

```
diff_occur_rs_gt_sp <-
read.csv("/home/yuri/liri/puzzle/comp_occur/all/3dp/diff_rs_gt_sp_all_chr.csv")
eqtls_occur_rs_gt_sp <- lapply(eqtls_vars_pos, inner_join, diff_occur_rs_gt_sp, by = c("chr",
"pos" = "position"))
```

```
eqtls_occur_rs_gt_sp <- Map(function(df, file) {
  if (nrow(df) > 0) {
    tissue_name <- gsub(".*?([A-Za-z_]+)\\.v10\\.eQTLs.*", "\\1", file)
    df$tissue <- tissue_name
  } else {
    df$tissue <- character(0) # Ensures an empty column for empty data frames
  }
  df
}, eqtls_occur_rs_gt_sp, eqtls_list)
```

```

eqtls_occure_rs_gt_sp <- do.call(rbind, eqtls_occure_rs_gt_sp)
eqtls_occure_rs_gt_sp$tissue <- gsub(".*(?:/|\\.)v10\\.eQTLs.*", "\\1",
eqtls_occure_rs_gt_sp$tissue)
colnames(eqtls_occure_rs_gt_sp) <- c("chr", "pos", "rsid", "tissue")

```

#Occurrences SP gt RS

```

diff_occure_sp_gt_rs <-
read.csv("/home/yuri/liri/puzzle/comp_occure/all/3dp/diff_sp_gt_rs_all_chr.csv")
eqtls_occure_sp_gt_rs <- lapply(eqtls_vars_pos, inner_join, diff_occure_sp_gt_rs, by = c("chr",
"pos" = "position"))
eqtls_occure_sp_gt_rs <- Map(function(df, file) {
  if (nrow(df) > 0) {
    tissue_name <- gsub(".*(?:/|\\.)v10\\.eQTLs.*", "\\1", file)
    df$tissue <- tissue_name
  } else {
    df$tissue <- character(0) # Ensures an empty column for empty data frames
  }
  df
}, eqtls_occure_sp_gt_rs, eqtls_list)
eqtls_occure_sp_gt_rs <- do.call(rbind, eqtls_occure_sp_gt_rs)
eqtls_occure_sp_gt_rs$tissue <- gsub(".*(?:/|\\.)v10\\.eQTLs.*", "\\1",
eqtls_occure_sp_gt_rs$tissue)
colnames(eqtls_occure_sp_gt_rs) <- c("chr", "pos", "rsid", "tissue")

```

#Allele freq differences

```

diff_freq <- read.csv("/home/yuri/liri/puzzle/comp_allele_freq/snvs_diff_all.csv")
diff_freq <- diff_freq[,c(2,3,7)]
eqtls_freq <- lapply(eqtls_vars_pos, inner_join, diff_freq, by = c("chr" = "CHR", "pos" = "BP"))
eqtls_freq <- Map(function(df, file) {
  if (nrow(df) > 0) {
    tissue_name <- gsub(".*(?:/|\\.)v10\\.eQTLs.*", "\\1", file)
    df$tissue <- tissue_name
  } else {
    df$tissue <- character(0) # Ensures an empty column for empty data frames
  }
  df
}, eqtls_freq, eqtls_list)
eqtls_freq <- do.call(rbind, eqtls_freq)
eqtls_freq $tissue <- gsub(".*(?:/|\\.)v10\\.eQTLs.*", "\\1", eqtls_freq$tissue)
colnames(eqtls_freq) <- c("chr", "pos", "rsid", "tissue")

```

#Number of eqtls for each comparison

```

eqtls_freq_number <- length(unique(eqtls_freq$rsid))
eqtls_occure_rs_gt_number <- length(unique(eqtls_occure_rs_gt_sp$rsid))
eqtls_occure_sp_gt_number <- length(unique(eqtls_occure_sp_gt_rs$rsid))

```

```

write.csv(eqtls_occur_rs_gt_sp, "/home/yuri/liri/puzzle/comp_occur/all/3dp/eqtls_rs_gt.csv",
row.names = FALSE, quote = FALSE)
write.csv(eqtls_occur_sp_gt_rs, "/home/yuri/liri/puzzle/comp_occur/all/3dp/eqtls_sp_gt.csv",
row.names = FALSE, quote = FALSE)
write.csv(eqtls_freq, "/home/yuri/liri/puzzle/comp_allele_freq/eqtls_diff.csv", row.names =
FALSE, quote = FALSE)

```

#Number of eqtls per tissue excluding variants that are eqtls for more than one gene in the tissue

```

eqtls_tissues_occur_rs_gt <- eqtls_occur_rs_gt_sp %>% distinct(rsid, tissue)
eqtls_tissues_occur_sp_gt <- eqtls_occur_sp_gt_rs %>% distinct(rsid, tissue)
eqtls_tissues_freq <- eqtls_freq %>% distinct(rsid, tissue)

```

```

n_eqtls_tissue_occur_rs_gt <- as.data.frame(table(eqtls_tissues_occur_rs_gt$tissue))
n_eqtls_tissue_occur_sp_gt <- as.data.frame(table(eqtls_tissues_occur_sp_gt$tissue))
n_eqtls_tissue_freq <- as.data.frame(table(eqtls_tissues_freq$tissue))

```

```

png("/home/yuri/liri/puzzle/comp_occur/all/3dp/n_eqtls_tissue_occur_rs_gt.png", width =
3800, height = 1080, res = 300)
ggplot(n_eqtls_tissue_occur_rs_gt, aes(x = Var1, y = Freq)) +
  geom_bar(stat = "identity", fill = "#FFB3BA") +
  theme_minimal() +
  labs(title = "Number of eQTLs per tissue (occurrences RS > SP)", x = "Tissue", y = "Number
of eQTLs") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) # Rotate x-axis labels for
readability
dev.off()

```

```

png("/home/yuri/liri/puzzle/comp_occur/all/3dp/n_eqtls_tissue_occur_sp_gt.png", width =
3800, height = 1080, res = 300)
ggplot(n_eqtls_tissue_occur_sp_gt, aes(x = Var1, y = Freq)) +
  geom_bar(stat = "identity", fill = "#B3D9FF") +
  theme_minimal() +
  labs(title = "Number of eQTLs per tissue (occurrences SP > RS)", x = "Tissue", y = "Number
of eQTLs") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) # Rotate x-axis labels for
readability
dev.off()

```

```

png("/home/yuri/liri/puzzle/comp_allele_freq/n_eqtls_tissue_freq.png", width = 3800, height =
1080, res = 300)
ggplot(n_eqtls_tissue_freq, aes(x = Var1, y = Freq)) +
  geom_bar(stat = "identity", fill = "#B3FFB3") +
  theme_minimal() +
  labs(title = "Number of eQTLs per tissue (frequency differences)", x = "Tissue", y = "Number
of eQTLs") +

```

```

    theme(axis.text.x = element_text(angle = 45, hjust = 1)) # Rotate x-axis labels for
readability
dev.off()

```

#Tissues with high amount of eqtls

```

mean_eqtls_tissue_rs_gt <- mean(n_eqtls_tissue_occur_rs_gt$Freq)
sd_eqtls_tissue_rs_gt <- sd(n_eqtls_tissue_occur_rs_gt$Freq)
alta_qtd_tec_rs_gt <- n_eqtls_tissue_occur_rs_gt$Freq > 3 * sd_eqtls_tissue_rs_gt
alta_qtd_tissue_rs <- n_eqtls_tissue_occur_rs_gt[alta_qtd_tec_rs_gt, ]

mean_eqtls_tissue_sp_gt <- mean(n_eqtls_tissue_occur_sp_gt$Freq)
sd_eqtls_tissue_sp_gt <- sd(n_eqtls_tissue_occur_sp_gt$Freq)
alta_qtd_tec_sp_gt <- n_eqtls_tissue_occur_sp_gt$Freq > 3 * sd_eqtls_tissue_sp_gt
alta_qtd_tissue_sp <- n_eqtls_tissue_occur_sp_gt[alta_qtd_tec_sp_gt, ]

mean_eqtls_tissue_freq <- mean(n_eqtls_tissue_freq$Freq)
sd_eqtls_tissue_freq <- sd(n_eqtls_tissue_freq$Freq)
alta_qtd_tec_freq <- n_eqtls_tissue_freq$Freq > 3 * sd_eqtls_tissue_freq
alta_qtd_tissue_freq <- n_eqtls_tissue_freq[alta_qtd_tec_freq, ]

```

## 08\_run\_genotype\_files\_nat.sh

```

#!/bin/bash
LOG_DIR="/home/yuri/liri/puzzle/scripts"
# Function to check available disk space
check_disk_space() {
    # Check if there is more than 1 GB of available space
    available_space=$(df "$VCF_DIR" | tail -1 | awk '{print $4}')
    if [ "$available_space" -lt 1048576 ]; then # 1 GB in KB
        echo "Error: Insufficient disk space. Stopping the process." >>
"$LOG_DIR/disk_space_error.log"
        exit 1
    fi
}

# Loop over chromosomes (1-22)
for CHR in {1..22}; do
    # Loop over states (you can modify this to match your state list or range)
    for STATE in "rs" "sp"; do
        # Before each iteration, check disk space
        check_disk_space

        # Call the genotype_files_nat.sh script
        echo "Processing chromosome $CHR and state $STATE..."
        bash /home/yuri/liri/puzzle/scripts/genotype_files_nat.sh "$CHR" "$STATE"

        # If no disk space issue, continue to the next state and chromosome
    done
done

```

```

        echo "Completed processing for chromosome $CHR and state $STATE" >>
"$LOG_DIR/processing.log"
    done
done

```

## 09\_get\_cutoff\_length.R

```

library(dplyr)
unfilt_fragments_dir <- "/home/yuri/liri/puzzle_sdumont/rs/nat/chr_info_unfilt" #change to sp if
doing it for another state
setwd(unfilt_fragments_dir)
files <- list.files(unfilt_fragments_dir, full.names = TRUE)
files <- files[1:22] #removing dirs and files that are not unfilt frags
files <- lapply(files, read.table, header = FALSE)
frag_sizes <- lapply(files, select, 5)
mode <- function(x) {mode <- rownames(as.data.frame((which(table(x) == max(table(x))))));
return(mode)}
modes <- lapply(frag_sizes, mode)
mean_modes <- function(x) {if(length(x) > 1) {x =
sum(as.numeric(x[1]),as.numeric(x[2])/length(x))} else {x = x}} #only valid if max length == 2,
which is my case
cutoff_length <- sapply(modes, mean_modes)
cutoff_length <- round(as.numeric(cutoff_length), 0)
write.table(cutoff_length, paste0(unfilt_fragments_dir, "/cutoff_length_rs.txt"), row.names = FALSE,
quote = FALSE)

```

## 10\_run.pipeline.sh

```

# This pipeline performs local ancestry inference for target populations using
ChromoPainterV2.
# It processes genomic data, infers local ancestry, and assembles ancestral haplotypes.
#
# Author: Xiaoxi Zhang
# Purpose: To process genomic data for local ancestry inference and assemble
ancestry-specific haplotypes.
# To help users run the pipeline more efficiently, we have removed the data preprocessing
section and the subsequent analysis for generating samples. If users require these steps,
please refer to our paper.
#
# Pipeline Steps:
# 1. Local ancestry inference for the target populations.
# 2. Determine which ancestries the recipient haplotypes copies from at each allele.
# 3. Data transpose and adding physical location information.
# 4. Extract the specified ancestral alleles from the selected samples.
# 5. Assemble ancestral haplotypes using extracted ancestral gene pools.
# 6. Generate ancestral individuals using assembled ancestral haplotypes.
# 7. Synthesize individual files into group files and remove missing variants.
#

```

```
# Input:
# - VCF files, genetic maps, and population-specific configuration files.
#
# Output:
# - Local ancestry results, integrated ancestral haplotypes, and assembled VCF files.
```

#This pipeline was modified by Liriel Almodobar to only contain steps 5, 6 and 7, in a loop for all chromosomes, removing intermediate files, automatizing cutoff\_length attribution, absolute paths only.

#"New" purpose: Construct non-admixed genomes from a given ancestry based on the tracts of that ancestry obtained from an admixed population.

#Q: Why not use steps 1 to 5? A: I already had results from RFMIX v1.0, so I created a pipeline to adapt them into input for step 5 forward.

#Q: Why is there a loop for "states"? A: I am trying to reassemble 100% native-american genomes from admixed individuals from 2 different brazilian states.

#Software

python=/opt/anaconda3/bin/python

#Python3

perl=/opt/anaconda3/bin/perl

#Perl v5 #LA: not sure it is being used for

my steps, check

bcftools=/usr/local/bin/bcftools

#BCFtools v1.14

#Working directory and scripts dir

WD=/workspace

SCRIPTS\_DIR=/workspace/scripts

TEMP\_DIR=\$WD/temp

mkdir -p \$TEMP\_DIR

#Initial parameters:

cutoff\_freq=\$1

echo "cutoff freq= " "\$cutoff\_freq"

for state in "rs"; do

#Define input and output\_directories

INPUT\_HAP\_DIR=\$WD/\$state/nat/chr\_info\_unfilt/haps\_simplified

INPUT\_VCF\_DIR=\$WD/\$state/nat/chr\_info\_unfilt/vcf\_nat

OUTPUT\_DIR=\$WD/\$state/nat/chr\_info\_unfilt/seq\_info/assembled

mkdir -p \$OUTPUT\_DIR

mkdir -p \$TEMP\_DIR

#Create array with cutoff\_lengths for all chrs (cutoff\_length is explained further below and needed for the steps)

array\_name=cutoffs\_\${state}

declare -n state\_array=\$array\_name

readarray state\_array < \$WD/\$state/nat/chr\_info\_unfilt/cutoff\_length\_\${state}.txt

```

#Determine number of haplotypes to be generated
if [ "$state" = "rs" ]; then
    n_haps=$(ls /workspace/output_collapse/chr_1 | grep C1 | wc -l) #LA:
output_collapse = 2 files per individual, matching mother and father haplotypes; C1 =
beginning of ID for rs individuals
    n_haps=$(awk "BEGIN {print $n_haps * 15.6 / 100}") #15.6 = native-american
ancestry % (ADMIXTURE)
else
    n_haps=$(ls $WD/workspace/chr_1 | grep C2 | wc -l) #C2: beginning of ID for
sp individuals
    n_haps=$(awk "BEGIN {print $n_haps * 11.6 / 100}") #LA: 11.6 =
native-american ancestry % (ADMIXTURE)
fi

echo "n_haps= " "$n_haps"

for chr in 21; do

    echo -e "${chr}\tchr${chr}" > $TEMP_DIR/rename_chr${chr}.txt #LA: necessary for
step 3 and dont wanna repeat it per state

    #Define cutoff_length
    cutoff_length=$(echo "${state_array[$chr]}") #LA: arrays are 0 indexed, but first line
from input is colname, so chr numbers match positions in the array
    echo "cutoff_length= " $cutoff_length
    echo "Step 1: Assemble haplotypes" "for chr " "$chr" "and state " "$state"
    # Assemble haplotypes using extracted gene pools

    #"inte.genome.cutoff.py" integrates gene pool across multiple files to generate an
haplotype.
    # python inte.genome.cutoff.py <Output haplotype> <Missing rate cutoff> <Length
cutoff> <Gene pool 1> ... < Gene pool n>
    # Length cutoff: The maximum length that can be extended at one time when
obtaining segments on a haplotype (e.g., 6000.0 bp).
    # Can input one or more gene pool files by appending them to the script parameters
in sequence.

    for i in $(seq 1 $n_haps) # Generate n_haps haplotypes
    do
        python $SCRIPTS_DIR/inte.genome.cutoff.py \
$TEMP_DIR/101.nat_${state}_chr${chr}.${i}.${cutoff_freq}.${cutoff_length}.txt.gz \
        ${cutoff_freq} \
        ${cutoff_length} \
        $INPUT_HAP_DIR/chr_${chr}_1_nat_${state}.hap.gz \
        $INPUT_HAP_DIR/chr_${chr}_2_nat_${state}.hap.gz
    done

```



```

echo "Step 2: Generate ancestral individuals using assembled ancestral haplotypes"

# Generate individuals from the assembled haplotypes
# "trans.to.vcf.py" converts two haplotypes into a VCF-format individual.
# python trans.to.vcf.py <Input gzipped VCF file> <haplotype 1> < haplotype 2>
<Sample ID> <Output individual>

    for i in $(seq 1 2 $n_haps)
    do
        j=$(( i + 1 ))
        #Combine two consecutively numbered haplotypes to generate a diploid
        $python $SCRIPTS_DIR/trans.to.vcf.py \
        $INPUT_VCF_DIR/dedup_chr_${chr}_nat_${state}.vcf.gz \

$TEMP_DIR/101.nat_${state}_chr${chr}.${i}.${cutoff_freq}.${cutoff_length}.txt.gz \

$TEMP_DIR/101.nat_${state}_chr${chr}.${j}.${cutoff_freq}.${cutoff_length}.txt.gz \
        $TEMP_DIR/201.nat_${state}_chr${chr}.${i}.${j} \

$TEMP_DIR/202.nat_${state}_chr${chr}.${i}.${j}.${cutoff_freq}.${cutoff_length}.vcf

        rm -f
$TEMP_DIR/101.nat_${state}_chr${chr}.${i}.${cutoff_freq}.${cutoff_length}.txt.gz
        rm -f
$TEMP_DIR/101.nat_${state}_chr${chr}.${j}.${cutoff_freq}.${cutoff_length}.txt.gz
        done

echo "Step 3: Synthesize individual files into group files and remove missing variants"

# Synthesize individual files into group files and remove missing variants
# Loop through chromosomes to merge individual VCF files
rm -f $TEMP_DIR/301.merge_${state}_chr${chr}.list.txt
# # Generate a list of VCF files for merging
    for i in $(seq 1 2 $n_haps)
    do
        j=$(( i + 1 ))
        echo
"$TEMP_DIR/202.nat_${state}_chr${chr}.${i}.${j}.${cutoff_freq}.${cutoff_length}.vcf.gz" >>
$TEMP_DIR/301.merge_${state}_chr${chr}.list.txt
        done
    # Merge the VCF files
    $bcftools merge -l $TEMP_DIR/301.merge_${state}_chr${chr}.list.txt -O z -o
$TEMP_DIR/302.nat_chr${chr}_${state}.${cutoff_freq}.${cutoff_length}.tmp.vcf.gz &&

```

```

tabix -f -p vcf
$TEMP_DIR/302.nat_chr${chr}_${state}.${cutoff_freq}.${cutoff_length}.tmp.vcf.gz
rm -f $TEMP_DIR/301.merge_${state}_chr${chr}.list.txt #LA: after the last iteration
there will still be one for chr 22
rm -f $TEMP_DIR/202*
# Rename chromosomes in the merged file.
$bcftools annotate --rename-chrs $TEMP_DIR/rename_chr${chr}.txt -O z -o
$OUTPUT_DIR/303.nat_chr${chr}_${state}.${cutoff_freq}.${cutoff_length}.vcf.gz
$TEMP_DIR/302.nat_chr${chr}_${state}.${cutoff_freq}.${cutoff_length}.tmp.vcf.gz && tabix
-f -p vcf $OUTPUT_DIR/303.nat_chr${chr}_${state}.${cutoff_freq}.${cutoff_length}.vcf.gz
rm -f
$TEMP_DIR/302.nat_chr${chr}_${state}.${cutoff_freq}.${cutoff_length}.tmp.vcf.gz*
rm -f $TEMP_DIR/rename_chr${chr}.txt

# Remove missing variants
$bcftools view -i "F_MISSING<0.001" --threads 10 -O z -o
$OUTPUT_DIR/304.nat_chr${chr}_${state}.${cutoff_freq}.${cutoff_length}.rm.missing.vcf.gz
$OUTPUT_DIR/303.nat_chr${chr}_${state}.${cutoff_freq}.${cutoff_length}.vcf.gz && tabix
-f -p vcf
$OUTPUT_DIR/304.nat_chr${chr}_${state}.${cutoff_freq}.${cutoff_length}.rm.missing.vcf.gz
initial_n_vars=$(bcftools view -H
$INPUT_VCF_DIR/dedup_chr_${chr}_nat_${state}.vcf.gz | wc -l)
cutoff_n_vars=$(awk "BEGIN {print $initial_n_vars * 0.8}")
final_n_vars=$(bcftools view -H
$OUTPUT_DIR/304.nat_chr${chr}_${state}.${cutoff_freq}.${cutoff_length}.rm.missing.vcf.gz |
wc -l)
if awk "BEGIN {exit !($final_n_vars < $cutoff_n_vars)}"; then
    echo "Final number of variants ($final_n_vars) is less than cutoff
($cutoff_n_vars). Removing files with cutoff_freq=${cutoff_freq}."
    rm -f "${OUTPUT_DIR}"/"${cutoff_freq}"*
    rm -f "${TEMP_DIR}"/"${cutoff_freq}"*
else
    echo "Final number of variants ($final_n_vars) meets or exceeds cutoff
($cutoff_n_vars). Files are retained."
fi
done
done

```

### **collapse\_ancestry.py**

```

__author__ = 'armartin'
#takes in RFMix output and creates collapsed 2 haploid bed files per individual

from datetime import datetime
import time
import argparse
from itertools import zip_longest
import gzip

```

```
import re
```

```
def parse_args():
```

```
    parser = argparse.ArgumentParser()
```

```
    parser.add_argument('--rfmix', help='path to RFMix Viterbi output, chr expected in filename',  
required=True)
```

```
    parser.add_argument('--snps_locations', help='path to snps_locations file required by RFMix,  
chr expected in filename', required=True)
```

```
    parser.add_argument('--fbk', default=None)
```

```
    parser.add_argument('--fbk_threshold', type=float, default = 0.9)
```

```
    parser.add_argument('--ind', help='Individual ID, must match a line in --ind_info option',  
required=True)
```

```
    parser.add_argument('--ind_info', help='Individual IDs listed in the order they appear in  
RFMix Viterbi output', required=True)
```

```
    parser.add_argument('--pop_labels', default='AFR, EUR, NAT',
```

```
                        help='comma-separated list of population labels in the order of rfmix populations  
(1 first, 2 second, and so on). Used in bed files and karyogram labels')
```

```
    parser.add_argument('--chr', help='chromosome number', type=int, required=True)
```

```
    parser.add_argument('--chrX', help='include chrX?', default=False, action="store_true")
```

```
    parser.add_argument('--out', help='prefix to bed file, _A.bed and _B.bed will be appended',  
required=True)
```

```
    args = parser.parse_args()
```

```
    return(args)
```

```
def grouper(n, iterable, fillvalue=None):
```

```
    "grouper(3, 'ABCDEFG', 'x') --> ABC DEF Gxx"
```

```
    args = [iter(iterable)] * n
```

```
    return zip_longest(fillvalue=fillvalue, *args)
```

```
def check_gt_posterior(fbk_max, fbk_threshold, index, add, hap_anc, line, current_anc):
```

```
    if fbk_max[index*2+add] >= fbk_threshold:
```

```
        hap_anc = myLine[index*2+add]
```

```
    else:
```

```
        hap_anc = -9
```

```
        current_anc[add] = -9
```

```
    return (current_anc, hap_anc)
```

```
def find_haplotype_bounds(index, add, pop_order, hap, npop):
```

```
    for chr in chrs:
```

```
        print (str(chr) + ' [' + datetime.fromtimestamp(time.time()).strftime('%Y-%m-%d  
%H:%M:%S') + ']')
```

```
        rfmix = open(re.sub(r'chr[X0-9]+', 'chr' + str(chr), args.rfmix))
```

```
        if args.fbk is not None:
```

```
            fbk_file = re.sub(r'chr[X0-9]+', 'chr' + str(chr), args.fbk)
```

```
            if fbk_file.endswith('.gz'):
```

```
                fbk = gzip.open(fbk_file)
```

```
            else:
```

```

    fbk = open(fbk_file)
    snp_file = re.sub(r'chr[X0-9]+', 'chr' + str(chr), args.snp_locations)
    snp_locations = open(snp_file)
    snp_map = open(snp_file.replace('snp_locations', 'map')) #map of physical position ->
genetic position

```

```

last_anc_pos_cm = [None, None, 0]

```

```

counter = 0
for line in rfmix:
    counter += 1
    myLine = line.strip().split()
    my_pos = snp_locations.readline().strip()
    my_map = snp_map.readline().strip().split()
    if args.fbk is not None:
        fbk_line = fbk.readline().strip().split()
        fbk_line = map(float, fbk_line)
        fbk_max = []
        for i in grouper(npop, fbk_line):
            fbk_max.append(max(i))
        if fbk_max[index*2+add] < args.fbk_threshold:
            myLine[index*2+add] = -9
    #fencepost for start of the chromosome
    if counter == 1:
        last_anc_pos_cm = [myLine[2*index + add], my_map[0], my_pos]
        post_anc_pos_cm = [myLine[2*index + add], my_map[0], my_pos]
        continue

```

```

#start regular iterations
current_anc_pos_cm = [myLine[2*index + add], my_map[0], my_pos]
if current_anc_pos_cm[0] == last_anc_pos_cm[0]:
    last_anc_pos_cm = current_anc_pos_cm
    continue
else:
    #we've reached the end of a region. Need to print.
    if last_anc_pos_cm[0] == -9:
        hap.write(str(chr) + '\t' + post_anc_pos_cm[1] + '\t' + last_anc_pos_cm[1] +
            '\tUNK\t' + post_anc_pos_cm[2] + '\t' + last_anc_pos_cm[2] + '\n')
    else:
        hap.write(str(chr) + '\t' + post_anc_pos_cm[1] + '\t' + last_anc_pos_cm[1] + '\t' +
            pop_order[int(last_anc_pos_cm[0])-1] + '\t' +
            post_anc_pos_cm[2] + '\t' + last_anc_pos_cm[2] + '\n')
    post_anc_pos_cm = current_anc_pos_cm

```

```

last_anc_pos_cm = current_anc_pos_cm

```

```

#last iteration, still need to print
if last_anc_pos_cm[0] == -9:

```

```

        hap.write(str(chr) + '\t' + post_anc_pos_cm[1] + '\t' + current_anc_pos_cm[1] +
                  '\tUNK\t' + post_anc_pos_cm[2] + '\t' + current_anc_pos_cm[2] + '\n')
    else:
        hap.write(str(chr) + '\t' + post_anc_pos_cm[1] + '\t' + current_anc_pos_cm[1] +
                  '\t' + pop_order[int(current_anc_pos_cm[0])-1] + '\t' +
                  post_anc_pos_cm[2] + '\t' + current_anc_pos_cm[2] + '\n')

def main(current_ind, index, pop_order):

    #open bed files (2 haplotypes per individual)
    hap_a = open(args.out + '_A.bed', 'w')
    hap_b = open(args.out + '_B.bed', 'w')

    find_haplotype_bounds(index, 0, pop_order, hap_a, npop)
    hap_a.close()

    find_haplotype_bounds(index, 1, pop_order, hap_b, npop)
    hap_b.close()

if __name__ == '__main__':
    #load parameters and files
    args = parse_args()
    pop_labels = args.pop_labels.split(',')
    npop = len(pop_labels)
    ind_info = open(args.ind_info)
    if args.ind is None:
        raise Exception('individual not set')
    current_ind = args.ind

    #reading order of individuals
    ind_list = []
    for line in ind_info:
        myLine = line.strip().split()
        ind_list.append(myLine[0])
    try:
        ind_index = ind_list.index(current_ind)
    except ValueError:
        raise Exception('Individual is not in the list provided')

    #set up chromosome variables
    chrs = range(args.chr, args.chr + 1)
    if args.chrX:
        chrs.append('X')

    print ('Starting [' + datetime.fromtimestamp(time.time()).strftime('%Y-%m-%d %H:%M:%S') +
    ']')
    main(current_ind, ind_index, pop_labels)

```

```
print ('Finished [' + datetime.fromtimestamp(time.time()).strftime('%Y-%m-%d %H:%M:%S')
+ '])
```

### **genotype\_files\_nat.sh**

```
#!/bin/bash
```

```
# Input, output, tools
```

```
CHR="$1"
```

```
STATE="$2"
```

```
WD="/home/yuri/liri/puzzle_sdumont"
```

```
UNFILT_FILE="$WD/$STATE/nat/chr_info_unfilt/chr_${CHR}_nat_${STATE}_unfilt.txt" #  
replace with path to file
```

```
HAPS_FILE="$WD/infos/haps_genos_header_${CHR}.txt" # replace with path to .haps
```

```
HAPS_DIR="$WD/$STATE/nat/chr_info_unfilt/haps_nat" #haps separated by chr from anc
```

```
HAPS_SIMPLIFIED_DIR=$HAPS_DIR/haps_simplified
```

```
VCF_DIR="$WD/$STATE/nat/chr_info_unfilt/vcf_nat" #vcf all samples all chr from anc
```

```
SHAPEIT=/home/yuri/Downloads/shapeit.v2.904.3.10.0-693.11.6.el7.x86_64/bin/shapeit #v2
```

```
BCFTOOLS=/usr/bin/bcftools
```

```
PLINK=/usr/local/bin/plink #v1.9
```

```
# Create output directories if they dont exist
```

```
mkdir -p "$HAPS_DIR"
```

```
mkdir -p "$VCF_DIR"
```

```
mkdir -p "$HAPS_SIMPLIFIED_DIR"
```

```
#Functions
```

```
haps_alleles_to_0_1() {
```

```
    local haps_alleles="$1" # Input file
```

```
    local haps_0_1="$2" # Output file
```

```
    # Use `awk` to process each row and reverse the modification
```

```
    awk '{
```

```
        fourth_col = $4
```

```
        fifth_col = $5
```

```
        reversed_line = $1 "\t" $2 "\t" $3 "\t" fourth_col "\t" fifth_col
```

```
        for (i = 6; i <= NF; i++) {
```

```
            if ($i == fourth_col) {
```

```
                reversed_line = reversed_line "\t0"
```

```
            } else if ($i == fifth_col) {
```

```
                reversed_line = reversed_line "\t1"
```

```
            } else {
```

```
                # If the value does not match fourth_col or fifth_col, keep it unchanged (optional)
```

```
                reversed_line = reversed_line "\t" $i
```

```
            }
```

```
        }
```

```
        print reversed_line
```

```
    } "$haps_alleles" > "$haps_0_1"
```

```
}
```

```
# Unique IDs without _A e _B
```

```
INDIVIDUALS=$(cut -f1 "$UNFILT_FILE" | sed 's/_.$//' | sort -u)
```

```
for ID in $INDIVIDUALS; do
```

```
    echo "Processando indivíduo: $ID"
```

```
    # Filter for NAT fragments
```

```
    grep -E "^${ID}_[AB]" "$UNFILT_FILE" > "$WD/${ID}_${CHR}_${STATE}_fragments.txt"
```

```
    # Extract coordinates
```

```
    awk '{print $2, $3, $4}' "$WD/${ID}_${CHR}_${STATE}_fragments.txt" >
"$WD/${ID}_${CHR}_${STATE}_coords.txt"
```

```
    rm "$WD/${ID}_${CHR}_${STATE}_fragments.txt"
```

```
    # Subset haps file for the haplotypes of one individual
```

```
    while read -r chrom initial_pos final_pos; do
```

```
        awk -v ip="$initial_pos" -v fp="$final_pos" '$3 >= ip && $3 <= fp {print $0}'
```

```
"$HAPS_FILE" >> "$WD/${ID}_${CHR}_${STATE}_subset.haps"
```

```
    done < "$WD/${ID}_${CHR}_${STATE}_coords.txt"
```

```
    rm "$WD/${ID}_${CHR}_${STATE}_coords.txt"
```

```
    # Keep only current individual in haps file
```

```
    col_number=$(head -n 1 "$HAPS_FILE" | tr '\t' '\n' | nl -v 6 | grep -i "$ID_A" | awk '{print $1}')  
    awk -v col_num="$col_number" -v OFS="\t" '{print $1, $2, $3, $4, $5, $col_num,
```

```
$(col_num+1)}' "$WD/${ID}_${CHR}_${STATE}_subset.haps" >
```

```
"$WD/${ID}_${CHR}_${STATE}_subset_final.txt"
```

```
    rm "$WD/${ID}_${CHR}_${STATE}_subset.haps"
```

```
    #Alleles in 0/1
```

```
    haps_alleles_to_0_1 "$WD/${ID}_${CHR}_${STATE}_subset_final.txt"
```

```
"$HAPS_DIR/${ID}_${CHR}_${STATE}_haps_nat.haps"
```

```
    rm "$WD/${ID}_${CHR}_${STATE}_subset_final.txt"
```

```
    #Create .sample file and convert haps to vcf
```

```
    haps_prefix="${HAPS_DIR}/${ID}_${CHR}_${STATE}_haps_nat"
```

```
    haps_file="${haps_prefix}.haps"
```

```
    # Define the .sample file
```

```
    sample_file="${haps_prefix}.sample"
```

```
    # Create the .sample file
```

```
    echo "Creating .sample file for $haps_file"
```

```

echo -e "ID_1\tID_2\tmissing" > "$sample_file"
echo -e "0\t0\t0" >> "$sample_file"
echo -e "${ID}\t${ID}\t0" >> "$sample_file"

# Define the output VCF file
output_vcf="${VCF_DIR}/${ID}_${CHR}_${STATE}_haps_nat.vcf"
sort -k3,3n "${haps_file}" -o "${haps_file}"

# Run Shapelt conversion
echo "Converting $haps_file to VCF format..."
$SHAPEIT -convert --input-haps "$haps_prefix" --output-vcf "$output_vcf"
rm "$HAPS_DIR/${ID}_${CHR}_${STATE}_haps_nat.haps"
rm "$HAPS_DIR/${ID}_${CHR}_${STATE}_haps_nat.sample"
done

#All vcfs of a chromosome into one
ulimit -n 10000 #increase limit of open files
#Compress
bgzip $VCF_DIR/*${CHR}_${STATE}_haps_nat.vcf

#Index
for VCF in $VCF_DIR/*${CHR}_${STATE}_haps_nat.vcf.gz; do $BCFTOOLS index
"$VCF"; done

#Merge
if [ "$STATE" = "rs" ]; then
    $BCFTOOLS merge -O z -o $VCF_DIR/all_C10_chr_${CHR}_nat_${STATE}.vcf.gz
    $VCF_DIR/C10*${CHR}_${STATE}_haps_nat.vcf.gz
    $BCFTOOLS merge -O z -o $VCF_DIR/all_C11_chr_${CHR}_nat_${STATE}.vcf.gz
    $VCF_DIR/C11*${CHR}_${STATE}_haps_nat.vcf.gz
    $BCFTOOLS index $VCF_DIR/all_C10_chr_${CHR}_nat_${STATE}.vcf.gz
    $BCFTOOLS index $VCF_DIR/all_C11_chr_${CHR}_nat_${STATE}.vcf.gz
    $BCFTOOLS merge -O z -o $VCF_DIR/chr_${CHR}_nat_${STATE}.vcf.gz
    $VCF_DIR/all_C*_chr_${CHR}_nat_${STATE}.vcf.gz
    $BCFTOOLS index $VCF_DIR/chr_${CHR}_nat_${STATE}.vcf.gz
    $BCFTOOLS norm -d all $VCF_DIR/chr_${CHR}_nat_${STATE}.vcf.gz -O z -o
    $VCF_DIR/dedup_chr_${CHR}_nat_${STATE}.vcf.gz
    $BCFTOOLS index $VCF_DIR/dedup_chr_${CHR}_nat_${STATE}.vcf.gz
fi
if [ "$STATE" = "sp" ]; then
    $BCFTOOLS merge -O z -o $VCF_DIR/all_C20_chr_${CHR}_nat_${STATE}.vcf.gz
    $VCF_DIR/C20*${CHR}_${STATE}_haps_nat.vcf.gz
    $BCFTOOLS merge -O z -o $VCF_DIR/all_C21_chr_${CHR}_nat_${STATE}.vcf.gz
    $VCF_DIR/C21*${CHR}_${STATE}_haps_nat.vcf.gz
    $BCFTOOLS index $VCF_DIR/all_C20_chr_${CHR}_nat_${STATE}.vcf.gz
    $BCFTOOLS index $VCF_DIR/all_C21_chr_${CHR}_nat_${STATE}.vcf.gz
    $BCFTOOLS merge -O z -o $VCF_DIR/chr_${CHR}_nat_${STATE}.vcf.gz
    $VCF_DIR/all_C*_chr_${CHR}_nat_${STATE}.vcf.gz

```



```

$BCFTOOLS index $VCF_DIR/chr_${CHR}_nat_${STATE}.vcf.gz
$BCFTOOLS norm -d all $VCF_DIR/chr_${CHR}_nat_${STATE}.vcf.gz -O z -o
$VCF_DIR/dedup_chr_${CHR}_nat_${STATE}.vcf.gz
$BCFTOOLS index $VCF_DIR/dedup_chr_${CHR}_nat_${STATE}.vcf.gz

fi

rm $VCF_DIR/C*
rm $VCF_DIR/all*
rm $VCF_DIR/chr*

#Haps with all the samples from the chromosome, already only segments from our desired
ancestry
$BCFTOOLS convert --hapsample "$HAPS_DIR/chr_${CHR}_nat_${STATE}"
"$VCF_DIR/dedup_chr_${CHR}_nat_${STATE}.vcf.gz"
#Get adapted haps to use as input for the pipeline
samples_file="$HAPS_DIR/chr_${CHR}_nat_${STATE}.samples"
out1_samples_file="$HAPS_SIMPLIFIED_DIR/chr_${CHR}_1_nat_${STATE}.samples"
out2_samples_file="$HAPS_SIMPLIFIED_DIR/chr_${CHR}_2_nat_${STATE}.samples"
out1_hap_file="$HAPS_SIMPLIFIED_DIR/chr_${CHR}_1_nat_${STATE}.hap"
out2_hap_file="$HAPS_SIMPLIFIED_DIR/chr_${CHR}_2_nat_${STATE}.hap"
zcat "$HAPS_DIR/chr_${CHR}_nat_${STATE}.hap.gz" | awk -v OFS="\t" '{ $2=$4=$5="";
$0=$0; print}' | sed 's#?#.#g' >
"$HAPS_SIMPLIFIED_DIR/temp_chr_${CHR}_nat_${STATE}.hap"

# Get the total number of lines, excluding the first two rows
total_lines=$(wc -l < "$samples_file")
remaining_lines=$((total_lines - 2))

# Calculate the midpoint, rounding up for the top half
midpoint=$(( (remaining_lines + 1) / 2 ))

# First two lines
head -n 2 "$samples_file" > "$out1_samples_file"
head -n 2 "$samples_file" > "$out2_samples_file"

# Split the remaining lines into top and bottom halves
tail -n +3 "$samples_file" | head -n "$midpoint" >> "$out1_samples_file"
tail -n +3 "$samples_file" | tail -n "$((remaining_lines - midpoint))" >> "$out2_samples_file"

# Subset haps for my new samples

# Define an associative array for sample files and their respective output files
declare -A sample_files
sample_files["$out1_samples_file"]="$out1_hap_file"
sample_files["$out2_samples_file"]="$out2_hap_file"

for sample_file in "${!sample_files[@]}"; do

```

```

# Extract the output file for the current sample file
output_file="${sample_files[$sample_file]}"

# Extract file prefix for naming intermediate files
file_prefix=$(basename "$sample_file" | sed 's/\.[^\.]*$//')

# Process the sample file
tail -n +3 "$sample_file" | grep -n C | sed 's#:0##g' | sed 's#_[^ ]*##g' | awk '{print $1, $2}' | \
    sed 'p' | awk 'NR%2{suffix="_1"} !(NR%2){suffix="_2"} {print $0 suffix}' | \
    awk '{if (NR % 2 == 0) $1 = ($1 * 2) + 2 ; else $1 = ($1 * 2) - 1 + 2} 1' >
"$HAPS_SIMPLIFIED_DIR/haps_cols_${CHR}_nat_${STATE}.txt"

# Generate haps_indexes file
awk '{print $1}' "$HAPS_SIMPLIFIED_DIR/haps_cols_${CHR}_nat_${STATE}.txt" >
"$HAPS_SIMPLIFIED_DIR/haps_indexes_${CHR}_nat_${STATE}"

# Declare an empty array
indexes=()

# Read the file line by line and populate the array
while IFS= read -r line; do
    indexes+=("$line")
done < "$HAPS_SIMPLIFIED_DIR/haps_indexes_${CHR}_nat_${STATE}"

# Process temp hap file using the indexes
intermediate_file="$HAPS_SIMPLIFIED_DIR/temp_processed_hap_${file_prefix}"
awk -v OFS="\t" -v indexes="${indexes[*]}" '{split(indexes, arr, " "); for (i in arr) printf "%s ",
$arr[i]; print ""}' \
    "$HAPS_SIMPLIFIED_DIR/temp_chr_${CHR}_nat_${STATE}.hap" >
"$HAPS_SIMPLIFIED_DIR/ind_cols_${file_prefix}"

# Extract the first two columns
awk '{print $1, $2}' "$HAPS_SIMPLIFIED_DIR/temp_chr_${CHR}_nat_${STATE}.hap" >
"$HAPS_SIMPLIFIED_DIR/first_haps_${file_prefix}"

# Generate column headers
geno_columns_file="$HAPS_SIMPLIFIED_DIR/geno_columns_${CHR}_${STATE}.txt"
awk '{print $2}' "$HAPS_SIMPLIFIED_DIR/haps_cols_${CHR}_nat_${STATE}.txt" | tr '\n' '\t'
> "$geno_columns_file"
header_file="$HAPS_SIMPLIFIED_DIR/header_${CHR}_${STATE}.txt"
echo -e "Chr\tPos\t$(cat $geno_columns_file)" > "$header_file"

# Combine header and intermediate file into final output
paste "$HAPS_SIMPLIFIED_DIR/first_haps_${file_prefix}"
"$HAPS_SIMPLIFIED_DIR/ind_cols_${file_prefix}" > "$intermediate_file"
cat "$header_file" "$intermediate_file" > "$output_file"
gzip "$output_file"

```

```

# Clean up intermediate files
rm "$HAPS_SIMPLIFIED_DIR/haps_indexes_${CHR}_nat_${STATE}"
rm "$HAPS_SIMPLIFIED_DIR/ind_cols_${file_prefix}"
rm "$HAPS_SIMPLIFIED_DIR/first_haps_${file_prefix}"
rm "$geno_columns_file"
rm "$header_file"
rm "$intermediate_file"
done
rm "$HAPS_SIMPLIFIED_DIR/temp_chr_${CHR}_nat_${STATE}.hap"
rm "$HAPS_SIMPLIFIED_DIR/haps_cols_${CHR}_nat_${STATE}.txt"
echo "fim"

```

### **inte.genome.cutoff.py**

```

"""

```

This script integrates ancestral gene pool across multiple files to generate an ancestral haplotype.

Author: Xiaoxi Zhang

Purpose: Integrate ancestral gene pools and filter variants based on user-defined cutoffs to generate an ancestral haplotype.

Input:

1. Output ancestral haplotype (gzipped).
2. Missing rate cutoff: Minimum missing rate (e.g., 0.2).
3. Length cutoff: The maximum length that can be extended at one time when obtaining ancestral segments on a haplotype (e.g., 10000.0 bp).
4. Input files: One or more gzipped files containing ancestral gene pool.

Output:

- A gzipped file containing filtered genomic data based on the specified cutoffs.

```

"""

```

```

import sys,os,zip,random
# Input parameters
outfile = sys.argv[1]      # Output file path
cutoff_freq = float(sys.argv[2]) # Missing rate cutoff (e.g., 0.2)
cutoff_length = float(sys.argv[3]) # Length cutoff (e.g., 10000.0)
infiles = sys.argv[4:]     # Input gzipped ancestral gene pool files

# Load and integrate ancestral gene pool from input files
info = [] # List to store integrated ancestral gene pool
for infile in infiles:
    k = 0
    f = gzip.open( infile, 'rt' )
    f.readline() # Skip the header line
    for line in f:

```

```

        y = line.split()
        chr, pos, genos = y[0], y[1], y[2:] # Chromosome, position, genotypes
        if k + 1 > len(info):
            info.append( [ chr, pos, ''.join(genos) ] )
            k += 1
        else:
            info[k][2] = info[k][2] + '|' + ''.join(genos) # Append genotypes
            k += 1
    f.close()

# Function to handle random selection for missing data
def deal_one_row(point, n, slgeno):
    """
    Randomly select a valid allele from the genotype list.
    If all genotypes are missing, return '.' and a random index.
    """
    d = { point: "" }
    while len(d) < n:
        t = random.randint( 0, n-1 )
        if t not in d:
            if slgeno[t] != '.':
                return slgeno[t], t
            else:
                d[t] = ""
        else:
            continue
    return '.', random.randint( 0, n-1 )

# Function to judge if a row passes the frequency cutoff
def judge( li ):
    """
    Check if the frequency of non-missing genotypes meets the cutoff.
    """
    kl = 0
    for i in range( len(li) ):
        if li[i] != '.':
            kl += 1
    if cutoff_freq <= float(kl) / len(li):
        return 'ok'
    else:
        return 'no'

# Write filtered data to output file
fout = gzip.open( outfile, 'wt' )
n = len(info[0][2].split('|')) # Number of samples
point = random.randint( 0, n-1 ) # Random starting point for selection
start = '.'
for i in range( len(info) ):

```

```

slgeno = info[i][2].split('|') # Split genotypes for the current row
res = judge( slgeno ) # Check frequency cutoff
if res != 'ok': # If frequency cutoff is not met
    point = random.randint( 0, n-1 )
    fout.write( info[i][0] + '\t' + info[i][1] + '\t' + '.' + '\n' )
    start = '.'
    continue
if slgeno[point] == '.': # Handle missing genotype at the current point
    fw, point = deal_one_row( point, n, slgeno )
    if fw == '.':
        fout.write( info[i][0] + '\t' + info[i][1] + '\t' + '.' + '\n' )
        start = '.'
    else:
        fout.write( info[i][0] + '\t' + info[i][1] + '\t' + fw + '\n' )
        start = info[i][1]
else: # If the genotype is valid
    result = ""
    if start == '.':
        result = 'ok'
    else:
        if int(info[i][1]) + 1 - int(start) > cutoff_length: # Check length cutoff
            result = 'no'
        if result == 'no':
            fw, point = deal_one_row( point, n, slgeno )
            if fw == '.':
                fout.write( info[i][0] + '\t' + info[i][1] + '\t' + fw + '\n' )
                start = '.'
            else:
                fout.write( info[i][0] + '\t' + info[i][1] + '\t' + fw + '\n' )
                start = info[i][1]
        else:
            fout.write( info[i][0] + '\t' + info[i][1] + '\t' + slgeno[point] + '\n' )
fout.close()

```

### **opt\_chr\_fragments\_plot.R**

```

# Load project environment
if (!requireNamespace("renv", quietly = TRUE)) {
    # Install renv and its dependencies
    install.packages("renv", dependencies = TRUE)
}
library(renv)
renv::restore()

# Load the required packages
library(karyoploteR)

# Set the working directory

```

```

wd <- "~/liri/teste_dir"

# Define the labels and corresponding colors
labels <- c("nat", "eur", "afr", "unk")
colors <- c("#6caf5e", "#7297b4", "#c05040", "#919191")

# Loop through the labels and chromosomes
for (i in 1:length(labels)) {
  label <- labels[i]
  color <- colors[i]
  for (chr in 1:22) {
    label_dir <- file.path(wd, label, "chr_info_filt_largest_frag")
    graph_dir <- "frags_plots"
    if (!file.exists(file.path(label_dir, graph_dir))) {
      dir.create(file.path(label_dir, graph_dir))
    }
    file_name_prefix <- paste0("chr_", chr, "_", label)
    positions_file <- file.path(label_dir, paste0(file_name_prefix, "_sort_filt_size_gap.txt"))
    pdf_file <- file.path(label_dir, graph_dir, paste0(file_name_prefix, "_plot_fragments.pdf"))
    # Read the data file
    dados <- read.table(positions_file, header = FALSE, sep = "\t")
    positions <- dados[2:4]
    positions[, 1] <- paste0("chr", positions[, 1])
    # Extract the chromosome from your data (assuming all values in the second column are
    the same)
    chromosome <- positions[1, 1]
    # Start PDF device
    pdf(pdf_file)
    # Create a karyotype object for chromosome drawing with the extracted chromosome
    kp <- plotKaryotype(genome = "hg38", chromosomes = chromosome)
    kpAddLabels(kp, labels = paste("frag", label))
    # Add fragments to the plot
    kpPlotRegions(kp, data = positions, col=color, border=darker(color),
    avoid.overlapping=TRUE)
    # End PDF device
    dev.off()
  }
}
# Save current environment
renv::snapshot()

```

## renv.lock

#Arquivo, nomeie como renv.lock e coloque o texto abaixo

```

{
  "R": {
    "Version": "4.3.0",

```

```

"Repositories": [
  {
    "Name": "CRAN",
    "URL": "https://cloud.r-project.org"
  }
],
"Bioconductor": {
  "Version": "3.17"
},
"Packages": {
  "AnnotationDbi": {
    "Package": "AnnotationDbi",
    "Version": "1.62.1",
    "Source": "Bioconductor",
    "git_url": "https://git.bioconductor.org/packages/AnnotationDbi",
    "git_branch": "RELEASE_3_17",
    "git_last_commit": "21f8479",
    "git_last_commit_date": "2023-05-01",
    "Requirements": [
      "Biobase",
      "BiocGenerics",
      "DBI",
      "IRanges",
      "KEGGREST",
      "R",
      "RSQLite",
      "S4Vectors",
      "methods",
      "stats",
      "stats4"
    ],
    "Hash": "23a8bf8b555ec9e4d20c8a0bec495786"
  },
  "AnnotationFilter": {
    "Package": "AnnotationFilter",
    "Version": "1.24.0",
    "Source": "Bioconductor",
    "git_url": "https://git.bioconductor.org/packages/AnnotationFilter",
    "git_branch": "RELEASE_3_17",
    "git_last_commit": "172d9c1",
    "git_last_commit_date": "2023-04-25",
    "Requirements": [
      "GenomicRanges",
      "R",
      "lazyeval",
      "methods",
      "utils"
    ]
  }
}

```

```

],
  "Hash": "5ee6a8b3d2ebe0eff2149c526dda4b64"
},
"BH": {
  "Package": "BH",
  "Version": "1.81.0-1",
  "Source": "Repository",
  "Repository": "CRAN",
  "Hash": "68122010f01c4dcfbe58ce7112f2433d"
},
"BSgenome": {
  "Package": "BSgenome",
  "Version": "1.68.0",
  "Source": "Bioconductor",
  "git_url": "https://git.bioconductor.org/packages/BSgenome",
  "git_branch": "RELEASE_3_17",
  "git_last_commit": "c546020",
  "git_last_commit_date": "2023-04-25",
  "Requirements": [
    "BiocGenerics",
    "Biostrings",
    "GenomeInfoDb",
    "GenomicRanges",
    "IRanges",
    "R",
    "Rsamtools",
    "S4Vectors",
    "XVector",
    "matrixStats",
    "methods",
    "rtracklayer",
    "stats",
    "utils"
  ],
  "Hash": "9d7c3c9b904c28bc971ed29d3f3b445e"
},
"Biobase": {
  "Package": "Biobase",
  "Version": "2.60.0",
  "Source": "Bioconductor",
  "git_url": "https://git.bioconductor.org/packages/Biobase",
  "git_branch": "RELEASE_3_17",
  "git_last_commit": "8dc10d2",
  "git_last_commit_date": "2023-04-25",
  "Requirements": [
    "BiocGenerics",
    "R",
    "methods",

```



```

    "utils"
  ],
  "Hash": "ed269b250f5844d54dfdc7e749f901aa"
},
"BiocFileCache": {
  "Package": "BiocFileCache",
  "Version": "2.8.0",
  "Source": "Bioconductor",
  "git_url": "https://git.bioconductor.org/packages/BiocFileCache",
  "git_branch": "RELEASE_3_17",
  "git_last_commit": "d088b32",
  "git_last_commit_date": "2023-04-25",
  "Requirements": [
    "DBI",
    "R",
    "RSQLite",
    "curl",
    "dbplyr",
    "dplyr",
    "filelock",
    "httr",
    "methods",
    "stats",
    "utils"
  ],
  "Hash": "4fcebdf379954d5d1a3291006a93499b"
},
"BiocGenerics": {
  "Package": "BiocGenerics",
  "Version": "0.46.0",
  "Source": "Bioconductor",
  "git_url": "https://git.bioconductor.org/packages/BiocGenerics",
  "git_branch": "RELEASE_3_17",
  "git_last_commit": "a90f0c5",
  "git_last_commit_date": "2023-04-25",
  "Requirements": [
    "R",
    "graphics",
    "methods",
    "stats",
    "utils"
  ],
  "Hash": "c179ae59955c36f5d0068ed29ce832f7"
},
"BiocIO": {
  "Package": "BiocIO",
  "Version": "1.10.0",
  "Source": "Bioconductor",

```

```

"git_url": "https://git.bioconductor.org/packages/BiocIO",
"git_branch": "RELEASE_3_17",
"git_last_commit": "1368ff1",
"git_last_commit_date": "2023-04-25",
"Requirements": [
  "BiocGenerics",
  "R",
  "S4Vectors",
  "methods",
  "tools"
],
"Hash": "a236af72143f9023b2f9f5f8baa81712"
},
"BiocManager": {
  "Package": "BiocManager",
  "Version": "1.30.21",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "utils"
  ],
  "Hash": "be203e7eea75514bc1a41c1de39a9bb9"
},
"BiocParallel": {
  "Package": "BiocParallel",
  "Version": "1.34.1",
  "Source": "Bioconductor",
  "git_url": "https://git.bioconductor.org/packages/BiocParallel",
  "git_branch": "RELEASE_3_17",
  "git_last_commit": "be65913",
  "git_last_commit_date": "2023-05-05",
  "Requirements": [
    "BH",
    "R",
    "codetools",
    "cpp11",
    "futile.logger",
    "methods",
    "parallel",
    "snow",
    "stats",
    "utils"
  ],
  "Hash": "6e0d920ff5276a1ab9990c35eb1bcd9c"
},
"BiocVersion": {
  "Package": "BiocVersion",
  "Version": "3.17.1",

```

```

"Source": "Bioconductor",
"git_url": "https://git.bioconductor.org/packages/BiocVersion",
"git_branch": "master",
"git_last_commit": "a2d0c4c",
"git_last_commit_date": "2022-11-02",
"Requirements": [
  "R"
],
"Hash": "f7c0d5521799b7b0d0a211143ed0bfc"
},
"Biostrings": {
  "Package": "Biostrings",
  "Version": "2.68.1",
  "Source": "Bioconductor",
  "git_url": "https://git.bioconductor.org/packages/Biostrings",
  "git_branch": "RELEASE_3_17",
  "git_last_commit": "cfbb402",
  "git_last_commit_date": "2023-05-16",
  "Requirements": [
    "BiocGenerics",
    "GenomeInfoDb",
    "IRanges",
    "R",
    "S4Vectors",
    "XVector",
    "crayon",
    "grDevices",
    "graphics",
    "methods",
    "stats",
    "utils"
  ],
  "Hash": "838eef43ab267a7409d68ba0fa8da5fa"
},
"DBI": {
  "Package": "DBI",
  "Version": "1.1.3",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "methods"
  ],
  "Hash": "b2866e62bab9378c3cc9476a1954226b"
},
"DelayedArray": {
  "Package": "DelayedArray",
  "Version": "0.26.2",

```

```

"Source": "Bioconductor",
"git_url": "https://git.bioconductor.org/packages/DelayedArray",
"git_branch": "RELEASE_3_17",
"git_last_commit": "9836b18",
"git_last_commit_date": "2023-05-05",
"Requirements": [
  "BiocGenerics",
  "IRanges",
  "Matrix",
  "MatrixGenerics",
  "R",
  "S4Arrays",
  "S4Vectors",
  "methods",
  "stats",
  "stats4"
],
"Hash": "65d87f0cf7b1fca7ecfb0201c1427cd4"
},
"Formula": {
  "Package": "Formula",
  "Version": "1.2-5",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "stats"
  ],
  "Hash": "7a29697b75e027767a53fde6c903eca7"
},
"GenomeInfoDb": {
  "Package": "GenomeInfoDb",
  "Version": "1.36.0",
  "Source": "Bioconductor",
  "git_url": "https://git.bioconductor.org/packages/GenomeInfoDb",
  "git_branch": "RELEASE_3_17",
  "git_last_commit": "c380bb9",
  "git_last_commit_date": "2023-04-25",
  "Requirements": [
    "BiocGenerics",
    "GenomeInfoDbData",
    "IRanges",
    "R",
    "RCurl",
    "S4Vectors",
    "methods",
    "stats",
    "stats4",

```

```

    "utils"
  ],
  "Hash": "97c524e262f664c987d66ceccd48bcad"
},
"GenomeInfoDbData": {
  "Package": "GenomeInfoDbData",
  "Version": "1.2.10",
  "Source": "Bioconductor",
  "Requirements": [
    "R"
  ],
  "Hash": "56294b21068b8cb5db1c47d0a42f307b"
},
"GenomicAlignments": {
  "Package": "GenomicAlignments",
  "Version": "1.36.0",
  "Source": "Bioconductor",
  "git_url": "https://git.bioconductor.org/packages/GenomicAlignments",
  "git_branch": "RELEASE_3_17",
  "git_last_commit": "cdc1aa4",
  "git_last_commit_date": "2023-04-25",
  "Requirements": [
    "BiocGenerics",
    "BiocParallel",
    "Biostrings",
    "GenomeInfoDb",
    "GenomicRanges",
    "IRanges",
    "R",
    "Rsamtools",
    "S4Vectors",
    "SummarizedExperiment",
    "methods",
    "stats",
    "utils"
  ],
  "Hash": "21b603ae11c96c397db2231103e4d430"
},
"GenomicFeatures": {
  "Package": "GenomicFeatures",
  "Version": "1.52.0",
  "Source": "Bioconductor",
  "git_url": "https://git.bioconductor.org/packages/GenomicFeatures",
  "git_branch": "RELEASE_3_17",
  "git_last_commit": "207ff08",
  "git_last_commit_date": "2023-04-25",
  "Requirements": [
    "AnnotationDbi",

```

```

"Biobase",
"BiocGenerics",
"BiocIO",
"Biostrings",
"DBI",
"GenomeInfoDb",
"GenomicRanges",
"IRanges",
"R",
"RCurl",
"RSQLite",
"S4Vectors",
"XVector",
"biomaRt",
"methods",
"rtracklayer",
"stats",
"tools",
"utils"
],
"Hash": "e4d36666fd5f1120360bc83b255b49b8"
},
"GenomicRanges": {
  "Package": "GenomicRanges",
  "Version": "1.52.0",
  "Source": "Bioconductor",
  "git_url": "https://git.bioconductor.org/packages/GenomicRanges",
  "git_branch": "RELEASE_3_17",
  "git_last_commit": "883f125",
  "git_last_commit_date": "2023-04-25",
  "Requirements": [
    "BiocGenerics",
    "GenomeInfoDb",
    "IRanges",
    "R",
    "S4Vectors",
    "XVector",
    "methods",
    "stats",
    "stats4",
    "utils"
  ],
  "Hash": "dc2970e434666341650a7983435597bf"
},
"Hmisc": {
  "Package": "Hmisc",
  "Version": "5.1-0",
  "Source": "Repository",

```

```

"Repository": "CRAN",
"Requirements": [
  "Formula",
  "base64enc",
  "cluster",
  "colorspace",
  "data.table",
  "foreign",
  "ggplot2",
  "grid",
  "gridExtra",
  "gtable",
  "htmlTable",
  "htmltools",
  "knitr",
  "methods",
  "nnet",
  "rmarkdown",
  "rpart",
  "viridis"
],
"Hash": "58bdffbf73db41c8bf9e4e2329f92429"
},
"IRanges": {
  "Package": "IRanges",
  "Version": "2.34.0",
  "Source": "Bioconductor",
  "git_url": "https://git.bioconductor.org/packages/IRanges",
  "git_branch": "RELEASE_3_17",
  "git_last_commit": "dcddf93",
  "git_last_commit_date": "2023-04-25",
  "Requirements": [
    "BiocGenerics",
    "R",
    "S4Vectors",
    "methods",
    "stats",
    "stats4",
    "utils"
  ],
  "Hash": "71c1620f45c7d30cb83f260b0f58fa3d"
},
"KEGGREST": {
  "Package": "KEGGREST",
  "Version": "1.40.0",
  "Source": "Bioconductor",
  "git_url": "https://git.bioconductor.org/packages/KEGGREST",
  "git_branch": "RELEASE_3_17",

```

```

"git_last_commit": "1bdb0b5",
"git_last_commit_date": "2023-04-25",
"Requirements": [
  "Biostrings",
  "R",
  "httr",
  "methods",
  "png"
],
"Hash": "b5a4bf06281c694bc38412812c70b011"
},
"MASS": {
  "Package": "MASS",
  "Version": "7.3-59",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "grDevices",
    "graphics",
    "methods",
    "stats",
    "utils"
  ],
  "Hash": "26570ae748e78cb2b0f56019dd2ba354"
},
"Matrix": {
  "Package": "Matrix",
  "Version": "1.5-1",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "graphics",
    "grid",
    "lattice",
    "methods",
    "stats",
    "utils"
  ],
  "Hash": "539dc0c0c05636812f1080f473d2c177"
},
"MatrixGenerics": {
  "Package": "MatrixGenerics",
  "Version": "1.12.0",
  "Source": "Bioconductor",
  "git_url": "https://git.bioconductor.org/packages/MatrixGenerics",
  "git_branch": "RELEASE_3_17",

```



```

    "git_last_commit": "442fde2",
    "git_last_commit_date": "2023-04-25",
    "Requirements": [
      "matrixStats",
      "methods"
    ],
    "Hash": "9f9813ce7b2adab14337f5516e337b65"
  },
  "ProtGenerics": {
    "Package": "ProtGenerics",
    "Version": "1.32.0",
    "Source": "Bioconductor",
    "git_url": "https://git.bioconductor.org/packages/ProtGenerics",
    "git_branch": "RELEASE_3_17",
    "git_last_commit": "30c6458",
    "git_last_commit_date": "2023-04-25",
    "Requirements": [
      "methods"
    ],
    "Hash": "6c853f577f1f8878d9fae988a0b542cc"
  },
  "R6": {
    "Package": "R6",
    "Version": "2.5.1",
    "Source": "Repository",
    "Repository": "CRAN",
    "Requirements": [
      "R"
    ],
    "Hash": "470851b6d5d0ac559e9d01bb352b4021"
  },
  "RColorBrewer": {
    "Package": "RColorBrewer",
    "Version": "1.1-3",
    "Source": "Repository",
    "Repository": "CRAN",
    "Requirements": [
      "R"
    ],
    "Hash": "45f0398006e83a5b10b72a90663d8d8c"
  },
  "RCurl": {
    "Package": "RCurl",
    "Version": "1.98-1.12",
    "Source": "Repository",
    "Repository": "CRAN",
    "Requirements": [
      "R",

```

```

    "bitops",
    "methods"
  ],
  "Hash": "1d6ed2d006d483f31c6d5531f3a39923"
},
"RSQLite": {
  "Package": "RSQLite",
  "Version": "2.3.1",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "DBI",
    "R",
    "bit64",
    "blob",
    "cpp11",
    "memoise",
    "methods",
    "pkgconfig",
    "plogr"
  ],
  "Hash": "207c90cd5438a1f596da2cd54c606fee"
},
"Rcpp": {
  "Package": "Rcpp",
  "Version": "1.0.10",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "methods",
    "utils"
  ],
  "Hash": "e749cae40fa9ef469b6050959517453c"
},
"Rhtslib": {
  "Package": "Rhtslib",
  "Version": "2.2.0",
  "Source": "Bioconductor",
  "git_url": "https://git.bioconductor.org/packages/Rhtslib",
  "git_branch": "RELEASE_3_17",
  "git_last_commit": "ac2e2a6",
  "git_last_commit_date": "2023-04-25",
  "Requirements": [
    "zlibbioc"
  ],
  "Hash": "225ae2e63b9c94991ee76bd33601b275"
},
"Rsamtools": {

```

```

"Package": "Rsamtools",
"Version": "2.16.0",
"Source": "Bioconductor",
"git_url": "https://git.bioconductor.org/packages/Rsamtools",
"git_branch": "RELEASE_3_17",
"git_last_commit": "3eb6d03",
"git_last_commit_date": "2023-04-25",
"Requirements": [
  "BiocGenerics",
  "BiocParallel",
  "Biostrings",
  "GenomeInfoDb",
  "GenomicRanges",
  "IRanges",
  "R",
  "Rhtslib",
  "S4Vectors",
  "XVector",
  "bitops",
  "methods",
  "stats",
  "utils",
  "zlibbioc"
],
"Hash": "e84f23bbbd554c5354471458a687046f"
},
"S4Arrays": {
  "Package": "S4Arrays",
  "Version": "1.0.4",
  "Source": "Bioconductor",
  "git_url": "https://git.bioconductor.org/packages/S4Arrays",
  "git_branch": "RELEASE_3_17",
  "git_last_commit": "d8d6a3b",
  "git_last_commit_date": "2023-05-12",
  "Requirements": [
    "BiocGenerics",
    "IRanges",
    "Matrix",
    "R",
    "S4Vectors",
    "crayon",
    "methods",
    "stats"
  ],
  "Hash": "3be34103255923c2fbb4a98b4d7449b9"
},
"S4Vectors": {
  "Package": "S4Vectors",

```

```

"Version": "0.38.1",
"Source": "Bioconductor",
"git_url": "https://git.bioconductor.org/packages/S4Vectors",
"git_branch": "RELEASE_3_17",
"git_last_commit": "17a9d8c",
"git_last_commit_date": "2023-05-01",
"Requirements": [
  "BiocGenerics",
  "R",
  "methods",
  "stats",
  "stats4",
  "utils"
],
"Hash": "f0a46e23a2a7d6aa27e945faf7f242c7"
},
"SummarizedExperiment": {
  "Package": "SummarizedExperiment",
  "Version": "1.30.1",
  "Source": "Bioconductor",
  "git_url": "https://git.bioconductor.org/packages/SummarizedExperiment",
  "git_branch": "RELEASE_3_17",
  "git_last_commit": "d7b3ada",
  "git_last_commit_date": "2023-04-30",
  "Requirements": [
    "Biobase",
    "BiocGenerics",
    "DelayedArray",
    "GenomeInfoDb",
    "GenomicRanges",
    "IRanges",
    "Matrix",
    "MatrixGenerics",
    "R",
    "S4Arrays",
    "S4Vectors",
    "methods",
    "stats",
    "tools",
    "utils"
  ],
  "Hash": "3c5f6bad810c9950abee035fabaa0fdc"
},
"VariantAnnotation": {
  "Package": "VariantAnnotation",
  "Version": "1.46.0",
  "Source": "Bioconductor",
  "git_url": "https://git.bioconductor.org/packages/VariantAnnotation",

```

```

"git_branch": "RELEASE_3_17",
"git_last_commit": "80d43e0",
"git_last_commit_date": "2023-04-25",
"Requirements": [
  "AnnotationDbi",
  "BSgenome",
  "Biobase",
  "BiocGenerics",
  "Biostrings",
  "DBI",
  "GenomeInfoDb",
  "GenomicFeatures",
  "GenomicRanges",
  "IRanges",
  "MatrixGenerics",
  "R",
  "Rhtslib",
  "Rsamtools",
  "S4Vectors",
  "SummarizedExperiment",
  "XVector",
  "methods",
  "rtracklayer",
  "utils",
  "zlibbioc"
],
"Hash": "0370b868500a1e497628b238fa67d4fe"
},
"XML": {
  "Package": "XML",
  "Version": "3.99-0.14",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "methods",
    "utils"
  ],
  "Hash": "e5c8af79df616c135b21eae1dc6bc5c"
},
"XVector": {
  "Package": "XVector",
  "Version": "0.40.0",
  "Source": "Bioconductor",
  "git_url": "https://git.bioconductor.org/packages/XVector",
  "git_branch": "RELEASE_3_17",
  "git_last_commit": "875b4b4",
  "git_last_commit_date": "2023-04-25",

```

```

"Requirements": [
  "BiocGenerics",
  "IRanges",
  "R",
  "S4Vectors",
  "methods",
  "tools",
  "utils",
  "zlibbioc"
],
"Hash": "cc3048ef590a16ff55a5e3149d5e060b"
},
"askpass": {
  "Package": "askpass",
  "Version": "1.1",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "sys"
  ],
  "Hash": "e8a22846fff485f0be3770c2da758713"
},
"backports": {
  "Package": "backports",
  "Version": "1.4.1",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R"
  ],
  "Hash": "c39fbec8a30d23e721980b8afb31984c"
},
"bamsignals": {
  "Package": "bamsignals",
  "Version": "1.32.0",
  "Source": "Bioconductor",
  "git_url": "https://git.bioconductor.org/packages/bamsignals",
  "git_branch": "RELEASE_3_17",
  "git_last_commit": "34bfc4e",
  "git_last_commit_date": "2023-04-25",
  "Requirements": [
    "BiocGenerics",
    "GenomicRanges",
    "IRanges",
    "R",
    "Rcpp",
    "Rhtslib",
    "methods",

```

```

    "zlibbioc"
  ],
  "Hash": "99c04208122dcbd0df56b9f6cc605ee3"
},
"base64enc": {
  "Package": "base64enc",
  "Version": "0.1-3",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R"
  ],
  "Hash": "543776ae6848fde2f48ff3816d0628bc"
},
"bezier": {
  "Package": "bezier",
  "Version": "1.1.2",
  "Source": "Repository",
  "Repository": "CRAN",
  "Hash": "a02b18c7fe82856f3e9a7e2f2a643435"
},
"biomaRt": {
  "Package": "biomaRt",
  "Version": "2.56.0",
  "Source": "Bioconductor",
  "git_url": "https://git.bioconductor.org/packages/biomaRt",
  "git_branch": "RELEASE_3_17",
  "git_last_commit": "91ca89e",
  "git_last_commit_date": "2023-04-25",
  "Requirements": [
    "AnnotationDbi",
    "BiocFileCache",
    "XML",
    "digest",
    "httr",
    "methods",
    "progress",
    "rappdirs",
    "stringr",
    "utils",
    "xml2"
  ],
  "Hash": "51728b7b82b32decee5d5925c3965c06"
},
"biovizBase": {
  "Package": "biovizBase",
  "Version": "1.48.0",
  "Source": "Bioconductor",

```

```

"git_url": "https://git.bioconductor.org/packages/biovizBase",
"git_branch": "RELEASE_3_17",
"git_last_commit": "534db77",
"git_last_commit_date": "2023-04-25",
"Requirements": [
  "AnnotationDbi",
  "AnnotationFilter",
  "BiocGenerics",
  "Biostrings",
  "GenomeInfoDb",
  "GenomicAlignments",
  "GenomicFeatures",
  "GenomicRanges",
  "Hmisc",
  "IRanges",
  "R",
  "RColorBrewer",
  "Rsamtools",
  "S4Vectors",
  "SummarizedExperiment",
  "VariantAnnotation",
  "dichromat",
  "ensemldb",
  "grDevices",
  "methods",
  "rlang",
  "scales",
  "stats"
],
"Hash": "d9ae4ebb3dfc8781361dbdd7ab0ebb0a"
},
"bit": {
  "Package": "bit",
  "Version": "4.0.5",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R"
  ],
  "Hash": "d242abec29412ce988848d0294b208fd"
},
"bit64": {
  "Package": "bit64",
  "Version": "4.0.5",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",

```



```
    "bit",
    "methods",
    "stats",
    "utils"
  ],
  "Hash": "9fe98599ca456d6552421db0d6772d8f"
},
"bitops": {
  "Package": "bitops",
  "Version": "1.0-7",
  "Source": "Repository",
  "Repository": "CRAN",
  "Hash": "b7d8d8ee39869c18d8846a184dd8a1af"
},
"blob": {
  "Package": "blob",
  "Version": "1.2.4",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "methods",
    "rlang",
    "vctrs"
  ],
  "Hash": "40415719b5a479b87949f3aa0aee737c"
},
"bslib": {
  "Package": "bslib",
  "Version": "0.4.2",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "base64enc",
    "cachem",
    "grDevices",
    "htmltools",
    "jquerylib",
    "jsonlite",
    "memoise",
    "mime",
    "rlang",
    "sass"
  ],
  "Hash": "a7fbf03946ad741129dc81098722fca1"
},
"cachem": {
  "Package": "cachem",
```

```
"Version": "1.0.8",
"Source": "Repository",
"Repository": "CRAN",
"Requirements": [
  "fastmap",
  "rlang"
],
"Hash": "c35768291560ce302c0a6589f92e837d"
},
"checkmate": {
  "Package": "checkmate",
  "Version": "2.2.0",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "backports",
    "utils"
  ],
  "Hash": "ca9c113196136f4a9ca9ce6079c2c99e"
},
"cli": {
  "Package": "cli",
  "Version": "3.6.1",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "utils"
  ],
  "Hash": "89e6d8219950eac806ae0c489052048a"
},
"cluster": {
  "Package": "cluster",
  "Version": "2.1.4",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "grDevices",
    "graphics",
    "stats",
    "utils"
  ],
  "Hash": "5edbbabab6ce0bf7900a74fd4358628e"
},
"codetools": {
  "Package": "codetools",
```

```

"Version": "0.2-19",
"Source": "Repository",
"Repository": "CRAN",
"Requirements": [
  "R"
],
"Hash": "c089a619a7fae175d149d89164f8c7d8"
},
"colorspace": {
  "Package": "colorspace",
  "Version": "2.1-0",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "grDevices",
    "graphics",
    "methods",
    "stats"
  ],
  "Hash": "f20c47fd52fae58b4e377c37bb8c335b"
},
"cpp11": {
  "Package": "cpp11",
  "Version": "0.4.3",
  "Source": "Repository",
  "Repository": "CRAN",
  "Hash": "ed588261931ee3be2c700d22e94a29ab"
},
"crayon": {
  "Package": "crayon",
  "Version": "1.5.2",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "grDevices",
    "methods",
    "utils"
  ],
  "Hash": "e8a1e41acf02548751f45c718d55aa6a"
},
"curl": {
  "Package": "curl",
  "Version": "5.0.0",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R"
  ]
}

```

```

    ],
    "Hash": "e4f97056611e8e6b8b852d13b7400cf1"
  },
  "data.table": {
    "Package": "data.table",
    "Version": "1.14.8",
    "Source": "Repository",
    "Repository": "CRAN",
    "Requirements": [
      "R",
      "methods"
    ],
    "Hash": "b4c06e554f33344e044ccd7fdca750a9"
  },
  "dbplyr": {
    "Package": "dbplyr",
    "Version": "2.3.2",
    "Source": "Repository",
    "Repository": "CRAN",
    "Requirements": [
      "DBI",
      "R",
      "R6",
      "blob",
      "cli",
      "dplyr",
      "glue",
      "lifecycle",
      "magrittr",
      "methods",
      "pillar",
      "purrr",
      "rlang",
      "tibble",
      "tidyr",
      "tidyselect",
      "utils",
      "vctrs",
      "withr"
    ],
    "Hash": "d24305b92db333726aed162a2c23a147"
  },
  "dichromat": {
    "Package": "dichromat",
    "Version": "2.0-0.1",
    "Source": "Repository",
    "Repository": "CRAN",
    "Requirements": [

```

```

    "R",
    "stats"
  ],
  "Hash": "16e66f2a483e124af5fc6582d26005f7"
},
"digest": {
  "Package": "digest",
  "Version": "0.6.31",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "utils"
  ],
  "Hash": "8b708f296afd9ae69f450f9640be8990"
},
"dplyr": {
  "Package": "dplyr",
  "Version": "1.1.2",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "R6",
    "cli",
    "generics",
    "glue",
    "lifecycle",
    "magrittr",
    "methods",
    "pillar",
    "rlang",
    "tibble",
    "tidyselect",
    "utils",
    "vctrs"
  ],
  "Hash": "dea6970ff715ca541c387de363ff405e"
},
"ellipsis": {
  "Package": "ellipsis",
  "Version": "0.3.2",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "rlang"
  ],

```

```

    "Hash": "bb0eec2fe32e88d9e2836c2f73ea2077"
  },
  "ensemldb": {
    "Package": "ensemldb",
    "Version": "2.24.0",
    "Source": "Bioconductor",
    "git_url": "https://git.bioconductor.org/packages/ensemldb",
    "git_branch": "RELEASE_3_17",
    "git_last_commit": "45a79a4",
    "git_last_commit_date": "2023-04-25",
    "Requirements": [
      "AnnotationDbi",
      "AnnotationFilter",
      "Biobase",
      "BiocGenerics",
      "Biostrings",
      "DBI",
      "GenomeInfoDb",
      "GenomicFeatures",
      "GenomicRanges",
      "IRanges",
      "ProtGenerics",
      "R",
      "RSQLite",
      "Rsamtools",
      "S4Vectors",
      "curl",
      "methods",
      "rtracklayer"
    ],
    "Hash": "cb99874dc074ad29fc691be022bb5974"
  },
  "evaluate": {
    "Package": "evaluate",
    "Version": "0.21",
    "Source": "Repository",
    "Repository": "CRAN",
    "Requirements": [
      "R",
      "methods"
    ],
    "Hash": "d59f3b464e8da1aef82dc04b588b8dfb"
  },
  "fansi": {
    "Package": "fansi",
    "Version": "1.0.4",
    "Source": "Repository",
    "Repository": "CRAN",

```

```

"Requirements": [
  "R",
  "grDevices",
  "utils"
],
"Hash": "1d9e7ad3c8312a192dea7d3db0274fde"
},
"farver": {
  "Package": "farver",
  "Version": "2.1.1",
  "Source": "Repository",
  "Repository": "CRAN",
  "Hash": "8106d78941f34855c440ddb946b8f7a5"
},
"fastmap": {
  "Package": "fastmap",
  "Version": "1.1.1",
  "Source": "Repository",
  "Repository": "CRAN",
  "Hash": "f7736a18de97dea803bde0a2daaafb27"
},
"filelock": {
  "Package": "filelock",
  "Version": "1.0.2",
  "Source": "Repository",
  "Repository": "CRAN",
  "Hash": "38ec653c2613bed60052ba3787bd8a2c"
},
"fontawesome": {
  "Package": "fontawesome",
  "Version": "0.5.1",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "htmltools",
    "rlang"
  ],
  "Hash": "1e22b8cabbad1eae951a75e9f8b52378"
},
"foreign": {
  "Package": "foreign",
  "Version": "0.8-82",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "methods",

```

```

    "stats",
    "utils"
  ],
  "Hash": "32b25c97ce306a760c4d9f787991b5d9"
},
"formatR": {
  "Package": "formatR",
  "Version": "1.14",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R"
  ],
  "Hash": "63cb26d12517c7863f5abb006c5e0f25"
},
"fs": {
  "Package": "fs",
  "Version": "1.6.2",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "methods"
  ],
  "Hash": "94af08e0aa9675a16fadbb3aaaa90d2a"
},
"futile.logger": {
  "Package": "futile.logger",
  "Version": "1.4.3",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "futile.options",
    "lambda.r",
    "utils"
  ],
  "Hash": "99f0ace8c05ec7d3683d27083c4f1e7e"
},
"futile.options": {
  "Package": "futile.options",
  "Version": "1.0.1",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R"
  ],
  "Hash": "0d9bf02413ddc2bbe8da9ce369dcdd2b"
}

```



```
},
"generics": {
  "Package": "generics",
  "Version": "0.1.3",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "methods"
  ],
  "Hash": "15e9634c0fcd294799e9b2e929ed1b86"
},
"ggplot2": {
  "Package": "ggplot2",
  "Version": "3.4.2",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "MASS",
    "R",
    "cli",
    "glue",
    "grDevices",
    "grid",
    "gtable",
    "isoband",
    "lifecycle",
    "mgcv",
    "rlang",
    "scales",
    "stats",
    "tibble",
    "vctrs",
    "withr"
  ],
  "Hash": "3a147ee02e85a8941aad9909f1b43b7b"
},
"glue": {
  "Package": "glue",
  "Version": "1.6.2",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "methods"
  ],
  "Hash": "4f2596dfb05dac67b9dc558e5c6fba2e"
},
```

```
"gridExtra": {
  "Package": "gridExtra",
  "Version": "2.3",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "grDevices",
    "graphics",
    "grid",
    "gtable",
    "utils"
  ],
  "Hash": "7d7f283939f563670a697165b2cf5560"
},
"gtable": {
  "Package": "gtable",
  "Version": "0.3.3",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "cli",
    "glue",
    "grid",
    "lifecycle",
    "rlang"
  ],
  "Hash": "b44addadb528a0d227794121c00572a0"
},
"highr": {
  "Package": "highr",
  "Version": "0.10",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "xfun"
  ],
  "Hash": "06230136b2d2b9ba5805e1963fa6e890"
},
"hms": {
  "Package": "hms",
  "Version": "1.1.3",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "lifecycle",
    "methods",
```

```

    "pkgconfig",
    "rlang",
    "vctrs"
  ],
  "Hash": "b59377caa7ed00fa41808342002138f9"
},
"htmlTable": {
  "Package": "htmlTable",
  "Version": "2.4.1",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "checkmate",
    "htmltools",
    "htmlwidgets",
    "knitr",
    "magrittr",
    "methods",
    "rstudioapi",
    "stringr"
  ],
  "Hash": "d5b485330dcfe241b50db157bf898e97"
},
"htmltools": {
  "Package": "htmltools",
  "Version": "0.5.5",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "base64enc",
    "digest",
    "ellipsis",
    "fastmap",
    "grDevices",
    "rlang",
    "utils"
  ],
  "Hash": "ba0240784ad50a62165058a27459304a"
},
"htmlwidgets": {
  "Package": "htmlwidgets",
  "Version": "1.6.2",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "grDevices",
    "htmltools",

```

```

    "jsonlite",
    "knitr",
    "rmarkdown",
    "yaml"
  ],
  "Hash": "a865aa85bcb2697f47505bfd70422471"
},
"httr": {
  "Package": "httr",
  "Version": "1.4.6",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "R6",
    "curl",
    "jsonlite",
    "mime",
    "openssl"
  ],
  "Hash": "7e5e3cbd2a7bc07880c94e22348fb661"
},
"isoband": {
  "Package": "isoband",
  "Version": "0.2.7",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "grid",
    "utils"
  ],
  "Hash": "0080607b4a1a7b28979aecef976d8bc2"
},
"jquerylib": {
  "Package": "jquerylib",
  "Version": "0.1.4",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "htmltools"
  ],
  "Hash": "5aab57a3bd297eee1c1d862735972182"
},
"jsonlite": {
  "Package": "jsonlite",
  "Version": "1.8.4",
  "Source": "Repository",
  "Repository": "CRAN",

```

```

"Requirements": [
  "methods"
],
"Hash": "a4269a09a9b865579b2635c77e572374"
},
"karyoploteR": {
  "Package": "karyoploteR",
  "Version": "1.26.0",
  "Source": "Bioconductor",
  "git_url": "https://git.bioconductor.org/packages/karyoploteR",
  "git_branch": "RELEASE_3_17",
  "git_last_commit": "473f8eb",
  "git_last_commit_date": "2023-04-25",
  "Requirements": [
    "AnnotationDbi",
    "GenomeInfoDb",
    "GenomicFeatures",
    "GenomicRanges",
    "IRanges",
    "R",
    "Rsamtools",
    "S4Vectors",
    "VariantAnnotation",
    "bamsignals",
    "bezier",
    "biovizBase",
    "digest",
    "grDevices",
    "graphics",
    "memoise",
    "methods",
    "regioneR",
    "rtracklayer",
    "stats"
  ],
  "Hash": "5f315dd86553dc5798c1135dac89b695"
},
"knitr": {
  "Package": "knitr",
  "Version": "1.42",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "evaluate",
    "highr",
    "methods",
    "tools",

```

```
    "xfun",
    "yaml"
  ],
  "Hash": "8329a9bcc82943c8069104d4be3ee22d"
},
"labeling": {
  "Package": "labeling",
  "Version": "0.4.2",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "graphics",
    "stats"
  ],
  "Hash": "3d5108641f47470611a32d0bdf357a72"
},
"lambda.r": {
  "Package": "lambda.r",
  "Version": "1.2.4",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "formatR"
  ],
  "Hash": "b1e925c4b9ffeb901bacf812cbe9a6ad"
},
"lattice": {
  "Package": "lattice",
  "Version": "0.21-8",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "grDevices",
    "graphics",
    "grid",
    "stats",
    "utils"
  ],
  "Hash": "0b8a6d63c8770f02a8b5635f3c431e6b"
},
"lazyeval": {
  "Package": "lazyeval",
  "Version": "0.2.2",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
```

```

    "R"
  ],
  "Hash": "d908914ae53b04d4c0c0fd72ecc35370"
},
"lifecycle": {
  "Package": "lifecycle",
  "Version": "1.0.3",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "cli",
    "glue",
    "rlang"
  ],
  "Hash": "001cecbeac1cff9301bdc3775ee46a86"
},
"magrittr": {
  "Package": "magrittr",
  "Version": "2.0.3",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R"
  ],
  "Hash": "7ce2733a9826b3aeb1775d56fd305472"
},
"matrixStats": {
  "Package": "matrixStats",
  "Version": "0.63.0",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R"
  ],
  "Hash": "57af0c3da1f1a50680b186591c3359af"
},
"memoise": {
  "Package": "memoise",
  "Version": "2.0.1",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "cachem",
    "rlang"
  ],
  "Hash": "e2817ccf4a065c5d9d7f2cfbe7c1d78c"
},

```

```

"mgcv": {
  "Package": "mgcv",
  "Version": "1.8-42",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "Matrix",
    "R",
    "graphics",
    "methods",
    "nlme",
    "splines",
    "stats",
    "utils"
  ],
  "Hash": "3460beba7ccc8946249ba35327ba902a"
},
"mime": {
  "Package": "mime",
  "Version": "0.12",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "tools"
  ],
  "Hash": "18e9c28c1d3ca1560ce30658b22ce104"
},
"munsell": {
  "Package": "munsell",
  "Version": "0.5.0",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "colorspace",
    "methods"
  ],
  "Hash": "6dfe8bf774944bd5595785e3229d8771"
},
"nlme": {
  "Package": "nlme",
  "Version": "3.1-162",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "graphics",
    "lattice",
    "stats",

```



```
    "utils"
  ],
  "Hash": "0984ce8da8da9ead8643c5cbbb60f83e"
},
"nnet": {
  "Package": "nnet",
  "Version": "7.3-18",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "stats",
    "utils"
  ],
  "Hash": "170da2130d5332bea7d6ede01875ba1d"
},
"openssl": {
  "Package": "openssl",
  "Version": "2.0.6",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "askpass"
  ],
  "Hash": "0f7cd2962e3044bb940cca4f4b5cecbe"
},
"pillar": {
  "Package": "pillar",
  "Version": "1.9.0",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "cli",
    "fansi",
    "glue",
    "lifecycle",
    "rlang",
    "utf8",
    "utils",
    "vctrs"
  ],
  "Hash": "15da5a8412f317beeee6175fbc76f4bb"
},
"pkgconfig": {
  "Package": "pkgconfig",
  "Version": "2.0.3",
  "Source": "Repository",
  "Repository": "CRAN",
```

```

    "Requirements": [
      "utils"
    ],
    "Hash": "01f28d4278f15c76cddbca05899c5d6f"
  },
  "plogr": {
    "Package": "plogr",
    "Version": "0.2.0",
    "Source": "Repository",
    "Repository": "CRAN",
    "Hash": "09eb987710984fc2905c7129c7d85e65"
  },
  "png": {
    "Package": "png",
    "Version": "0.1-8",
    "Source": "Repository",
    "Repository": "CRAN",
    "Requirements": [
      "R"
    ],
    "Hash": "bd54ba8a0a5faded999a7aab6e46b374"
  },
  "prettyunits": {
    "Package": "prettyunits",
    "Version": "1.1.1",
    "Source": "Repository",
    "Repository": "CRAN",
    "Hash": "95ef9167b75dde9d2ccc3c7528393e7e"
  },
  "progress": {
    "Package": "progress",
    "Version": "1.2.2",
    "Source": "Repository",
    "Repository": "CRAN",
    "Requirements": [
      "R6",
      "crayon",
      "hms",
      "prettyunits"
    ],
    "Hash": "14dc9f7a3c91ebb14ec5bb9208a07061"
  },
  "purrr": {
    "Package": "purrr",
    "Version": "1.0.1",
    "Source": "Repository",
    "Repository": "CRAN",
    "Requirements": [

```

```

    "R",
    "cli",
    "lifecycle",
    "magrittr",
    "rlang",
    "vctrs"
  ],
  "Hash": "d71c815267c640f17ddb7f7f16144b4bb"
},
"rappdirs": {
  "Package": "rappdirs",
  "Version": "0.3.3",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R"
  ],
  "Hash": "5e3c5dc0b071b21fa128676560dbe94d"
},
"regioneR": {
  "Package": "regioneR",
  "Version": "1.32.0",
  "Source": "Bioconductor",
  "git_url": "https://git.bioconductor.org/packages/regioneR",
  "git_branch": "RELEASE_3_17",
  "git_last_commit": "7d4c5c0",
  "git_last_commit_date": "2023-04-25",
  "Requirements": [
    "BSgenome",
    "Biostrings",
    "GenomeInfoDb",
    "GenomicRanges",
    "IRanges",
    "S4Vectors",
    "graphics",
    "memoise",
    "methods",
    "parallel",
    "rtracklayer",
    "stats",
    "tools",
    "utils"
  ],
  "Hash": "87c789c1e767df33772a1024fff94426"
},
"renv": {
  "Package": "renv",
  "Version": "0.17.3",

```

```

"Source": "Repository",
"Repository": "CRAN",
"Requirements": [
  "utils"
],
"Hash": "4543b8cd233ae25c6aba8548be9e747e"
},
"restfulr": {
  "Package": "restfulr",
  "Version": "0.0.15",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "RCurl",
    "S4Vectors",
    "XML",
    "methods",
    "rjson",
    "yaml"
  ],
  "Hash": "44651c1e68eda9d462610aca9f15a815"
},
"rjson": {
  "Package": "rjson",
  "Version": "0.2.21",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R"
  ],
  "Hash": "f9da75e6444e95a1baf8ca24909d63b9"
},
"rlang": {
  "Package": "rlang",
  "Version": "1.1.1",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "utils"
  ],
  "Hash": "a85c767b55f0bf9b7ad16c6d7baee5bb"
},
"rmarkdown": {
  "Package": "rmarkdown",
  "Version": "2.21",
  "Source": "Repository",

```

```

"Repository": "CRAN",
"Requirements": [
  "R",
  "bslib",
  "evaluate",
  "fontawesome",
  "htmltools",
  "jquerylib",
  "jsonlite",
  "knitr",
  "methods",
  "stringr",
  "tinytex",
  "tools",
  "utils",
  "xfun",
  "yaml"
],
"Hash": "493df4ae51e2e984952ea4d5c75786a3"
},
"rpart": {
  "Package": "rpart",
  "Version": "4.1.19",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "grDevices",
    "graphics",
    "stats"
  ],
  "Hash": "b3c892a81783376cc2204af0f5805a80"
},
"rstudioapi": {
  "Package": "rstudioapi",
  "Version": "0.14",
  "Source": "Repository",
  "Repository": "CRAN",
  "Hash": "690bd2acc42a9166ce34845884459320"
},
"rtracklayer": {
  "Package": "rtracklayer",
  "Version": "1.60.0",
  "Source": "Bioconductor",
  "git_url": "https://git.bioconductor.org/packages/rtracklayer",
  "git_branch": "RELEASE_3_17",
  "git_last_commit": "de35bc0",
  "git_last_commit_date": "2023-04-25",

```

```

"Requirements": [
  "BiocGenerics",
  "BiocIO",
  "Biostrings",
  "GenomeInfoDb",
  "GenomicAlignments",
  "GenomicRanges",
  "IRanges",
  "R",
  "RCurl",
  "Rsamtools",
  "S4Vectors",
  "XML",
  "XVector",
  "methods",
  "restfulr",
  "tools",
  "zlibbioc"
],
"Hash": "d6e636b707f7807823a0b554077eea60"
},
"sass": {
  "Package": "sass",
  "Version": "0.4.6",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R6",
    "fs",
    "htmltools",
    "rappdirs",
    "rlang"
  ],
  "Hash": "cc3ec7dd33982ef56570229b62d6388e"
},
"scales": {
  "Package": "scales",
  "Version": "1.2.1",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "R6",
    "RColorBrewer",
    "farver",
    "labeling",
    "lifecycle",
    "munsell",

```

```

    "rlang",
    "viridisLite"
  ],
  "Hash": "906cb23d2f1c5680b8ce439b44c6fa63"
},
"snow": {
  "Package": "snow",
  "Version": "0.4-4",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "utils"
  ],
  "Hash": "40b74690debd20c57d93d8c246b305d4"
},
"stringi": {
  "Package": "stringi",
  "Version": "1.7.12",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "stats",
    "tools",
    "utils"
  ],
  "Hash": "ca8bd84263c77310739d2cf64d84d7c9"
},
"stringr": {
  "Package": "stringr",
  "Version": "1.5.0",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "cli",
    "glue",
    "lifecycle",
    "magrittr",
    "rlang",
    "stringi",
    "vctrs"
  ],
  "Hash": "671a4d384ae9d32fc47a14e98bfa3dc8"
},
"sys": {
  "Package": "sys",

```

```
"Version": "3.4.1",
"Source": "Repository",
"Repository": "CRAN",
"Hash": "34c16f1ef796057bfa06d3f4ff818a5d"
},
"tibble": {
  "Package": "tibble",
  "Version": "3.2.1",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "fansi",
    "lifecycle",
    "magrittr",
    "methods",
    "pillar",
    "pkgconfig",
    "rlang",
    "utils",
    "vctrs"
  ],
  "Hash": "a84e2cc86d07289b3b6f5069df7a004c"
},
"tidyr": {
  "Package": "tidyr",
  "Version": "1.3.0",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "cli",
    "cpp11",
    "dplyr",
    "glue",
    "lifecycle",
    "magrittr",
    "purrr",
    "rlang",
    "stringr",
    "tibble",
    "tidyselect",
    "utils",
    "vctrs"
  ],
  "Hash": "e47debd7ce599b070c8e78e8ac0cfcf"
},
"tidyselect": {
```



```

"Package": "tidyselect",
"Version": "1.2.0",
"Source": "Repository",
"Repository": "CRAN",
"Requirements": [
  "R",
  "cli",
  "glue",
  "lifecycle",
  "rlang",
  "vctrs",
  "withr"
],
"Hash": "79540e5fcd9e0435af547d885f184fd5"
},
"tinytex": {
  "Package": "tinytex",
  "Version": "0.45",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "xfun"
  ],
  "Hash": "e4e357f28c2edff493936b6cb30c3d65"
},
"utf8": {
  "Package": "utf8",
  "Version": "1.2.3",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R"
  ],
  "Hash": "1fe17157424bb09c48a8b3b550c753bc"
},
"vctrs": {
  "Package": "vctrs",
  "Version": "0.6.2",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "cli",
    "glue",
    "lifecycle",
    "rlang"
  ],
  "Hash": "a745bda7aff4734c17294bb41d4e4607"

```

```
},
"viridis": {
  "Package": "viridis",
  "Version": "0.6.3",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "ggplot2",
    "gridExtra",
    "viridisLite"
  ],
  "Hash": "0d774f552202add033efc43a30293b3f"
},
"viridisLite": {
  "Package": "viridisLite",
  "Version": "0.4.2",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R"
  ],
  "Hash": "c826c7c4241b6fc89ff55aaea3fa7491"
},
"withr": {
  "Package": "withr",
  "Version": "2.5.0",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "grDevices",
    "graphics",
    "stats"
  ],
  "Hash": "c0e49a9760983e81e55cdd9be92e7182"
},
"xfun": {
  "Package": "xfun",
  "Version": "0.39",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "stats",
    "tools"
  ],
  "Hash": "8f56e9acb54fb525e66464d57ab58bcb"
},
```

```

"xml2": {
  "Package": "xml2",
  "Version": "1.3.4",
  "Source": "Repository",
  "Repository": "CRAN",
  "Requirements": [
    "R",
    "methods"
  ],
  "Hash": "7dc765ac9b909487326a7d471fdd3821"
},
"yaml": {
  "Package": "yaml",
  "Version": "2.3.7",
  "Source": "Repository",
  "Repository": "CRAN",
  "Hash": "0d0056cc5383fbc240ccd0cb584bf436"
},
"zlibbioc": {
  "Package": "zlibbioc",
  "Version": "1.46.0",
  "Source": "Bioconductor",
  "git_url": "https://git.bioconductor.org/packages/zlibbioc",
  "git_branch": "RELEASE_3_17",
  "git_last_commit": "f475457",
  "git_last_commit_date": "2023-04-25",
  "Hash": "20158ef5adb641f0b4e8d63136f0e870"
}
}
}

```

### **trans.to.vcf.py**

"""

This script converts two ancestral haplotypes into a VCF-format ancestral individual. It integrates reference and alternate allele information from input files and formats it for downstream analysis.

Author: Xiaoxi Zhang

Purpose: Generate an ancestral individual using two ancestral haplotypes.

Input:

1. invcf: A gzipped VCF file with reference and alternate allele information.
2. intxt1: A gzipped file containing the first ancestral haplotype.
3. intxt2: A gzipped file containing the second ancestral haplotype.
4. samID: Specify the name of the generated ancestral sample.
5. outfil: Output VCF file path.

Output:

- A compressed VCF file with genotype information.
- Associated index file for the VCF file.

"""

```
import sys, os, gzip
```

```
# Input and output file paths
```

```
invcf = sys.argv[1] # Input gzipped VCF file
```

```
intxt1 = sys.argv[2] # Ancestral haplotype 1
```

```
intxt2 = sys.argv[3] # Ancestral haplotype 2
```

```
samID = sys.argv[4] # Sample ID
```

```
outfil = sys.argv[5] # Output VCF file
```

```
# Step 1: Parse the reference and alternate alleles from the input VCF file
```

```
d = {} # Dictionary to store reference and alternate allele information
```

```
f = gzip.open( invcf, 'rt' )
```

```
for line in f:
```

```
    if line[0] == '#': # Skip header line
```

```
        continue
```

```
    y = line.split()
```

```
    chr, pos, ref, alt = y[0], y[1], y[3], y[4]
```

```
    if chr[:3] == 'chr': # Remove "chr" prefix if present
```

```
        chr = chr[3:]
```

```
    d[ chr + ':' + pos ] = [ ref, alt ]
```

```
f.close()
```

```
# Step 2: Add allele information from intxt1
```

```
f = gzip.open( intxt1, 'rt' )
```

```
for line in f:
```

```
    y = line.split()
```

```
    chr, pos, allele = y[0], y[1], y[2][0] # Extract chromosome, position, and allele
```

```
    d[ chr + ':' + pos ].append( allele )
```

```
f.close()
```

```
# Step 3: Add allele information from intxt2
```

```
f = gzip.open( intxt2, 'rt' )
```

```
for line in f:
```

```
    y = line.split()
```

```
    chr, pos, allele = y[0], y[1], y[2][0] # Extract chromosome, position, and allele
```

```
    d[ chr + ':' + pos ].append( allele )
```

```
f.close()
```

```
# Step 4: Write VCF output
```

```
fout = open( outfil, 'w' )
```

```
# Write VCF header
```

```
fout.write( '##fileformat=VCFv4.2\n##FILTER=<ID=PASS,Description="All filters
```

```
passed">\n##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">\n##contig
```

```

=<ID=1>\n###contig=<ID=2>\n###contig=<ID=3>\n###contig=<ID=4>\n###contig=<ID=5>\n###co
ntig=<ID=6>\n###contig=<ID=7>\n###contig=<ID=8>\n###contig=<ID=9>\n###contig=<ID=10>\n
###contig=<ID=11>\n###contig=<ID=12>\n###contig=<ID=13>\n###contig=<ID=14>\n###contig=<
ID=15>\n###contig=<ID=16>\n###contig=<ID=17>\n###contig=<ID=18>\n###contig=<ID=19>\n#
#contig=<ID=20>\n###contig=<ID=21>\n###contig=<ID=22>\n###contig=<ID=X>\n###contig=<ID
=Y>\n###contig=<ID=MT>\n###contig=<ID=M>\n' )
fout.write( '\t'.join( '#CHROM POS ID REF ALT QUAL FILTER INFO
FORMAT'.split() + [ samID ] ) + '\n' )
# Write variant information
for dd in d:
    chr, pos, ref, alt, a1, a2 = dd.split(':')[0], dd.split(':')[1], d[dd][0], d[dd][1], d[dd][2],
d[dd][3] # Extract chromosome, position and alleles
    fout.write( '\t'.join( [ chr, pos, '.', ref, alt, '.', 'PASS', '.', 'GT', a1 + '|' + a2 ] ) + '\n' )
fout.close()

# Step 5: Compress and index the VCF file
os.system( 'bgzip -f ' + outfil ) # Compress the VCF file
os.system( 'tabix -f -p vcf ' + outfil + '.gz' ) # Create an index for the VCF file

```